Code explanations

Python from zero and the three Colab notebooks, cell by cell

CNN-LSTM Based Spatiotemporal Flood Evolution Forecasting · Anhad Mahajan and Aman Sumeet Saxena · Deep Learning CA3

1 Python from zero: only what our notebooks use

You only need to read one line and say in your own words what it does. Our three notebooks use a small part of Python, listed here.

1.1 Colab

Colab is a free notebook in the browser made of **cells**: text cells (headings) and code cells (Python). Run a code cell with the play button or Shift+Enter. Its output stays saved under it. `!pip install -q rasterio` (a line starting with `!` is a computer command, not Python) installs a library. `drive.mount("/content/drive")` connects Google Drive, so Drive is the folder `/content/drive/MyDrive` where data and saved models live. `/content` is a temporary disk wiped at the end of the session. `userdata.get("HF_TOKEN")` reads a secret key (Hugging Face token). A **GPU runtime** (T4) makes networks much faster.

1.2 The very basics

A line starting with `#` is a comment, ignored by Python. A **variable** is a name for a value, and = means "store under this name". Names in CAPITALS (`TILE_SIZE`) are settings chosen once at the top, only a habit.

```
step_hours = 0.5       # decimal number
region = "UK"          # text in quotes
train_new = False      # True or False
print("Region:", region)
```

`+ - * /` are the usual operations, `**` is power (`2**3` is 8), `//` drops decimals (`7//2` is 3), `%` is the remainder (`7%2` is 1). The remainder picks every 30-minute map: `seconds % 1800 == 0` is True for a file named 1800 and False for 2100. Comparisons `== != < > <= >=` give True or False, joined by `and`, `or`, `not`. `None` means nothing. `1e-6` is 0.000001 and the notebooks use `-1e30` for "no data". Handy: `len`, `sum`, `max`, `min`, `abs`, `round`, `int`, `str`.

1.3 Lists and tuples

A **list** uses square brackets and counts from **0**. With `seconds = [0,300,600,...,3600]`: `seconds[0]` is 0, `seconds[-1]` is 3600 (the last), `seconds[2:5]` is 600, 900, 1200 (the end is excluded), `seconds[::6]` is every sixth item, [0, 1800, 3600]. The exploration notebook uses this to keep one depth map in six: `get_depth_files(dataset)[::DEPTH_MAPS_PER_RAIN_MAP]`. `files.append(x)` adds an item and `files.sort(key=get_seconds)` orders by the number the function gives. A **tuple** such as `(top, left)` is a fixed bundle. A **list comprehension** builds a list in one line: `[f for f in files if get_seconds(f) % 1800 == 0]` means "every f in files whose time is on a 30-minute mark".

1.4 Dictionaries

A **dictionary** holds name and value pairs in curly brackets: `info = {"region": "UK", "row_offset": 5}`, then `info["row_offset"]` is 5. `DATASETS` is a dictionary of dictionaries, one entry per dataset (folder, square size, scale, offsets). `for dataset, info in DATASETS.items():` walks through them. A list of dictionaries (one per row) is how tables are built (Section 1.12).

1.5 Loops and decisions

A **for loop** repeats its **indented** lines for each item. Indentation (4 spaces) shows what belongs to a loop, `if` or `def`, and the line before it ends with a colon.

```
for step in range(3):                # 0, 1, 2 (the stop number is excluded)
    print(step)
if share_now < 0.01 and share_forecast < 0.01:
    answer = "None"
elif share_forecast >= share_now:
    answer = "at least the current level"
else:
    answer = "keep the network's answer"    # only the first true branch runs
```

`range(0,10,3)` is 0, 3, 6, 9. `enumerate(files)` gives a counter and the item. `zip(a, b)` walks two lists together. `while len(inputs) < count:` repeats while true (the tile maker uses it until it has 4,000 tiles). `break` leaves a loop, `continue` skips to the next round.

1.6 Functions

A **function** is named code, made with `def` and used by calling its name. The names in brackets are **parameters**, and `return` hands the answer back.

```
def severity_class(share):
    return int(np.digitize(share, [0.05, 0.15, 0.30]))
severity_class(0.20)     # 2, which means Moderate
```

A function can return two things (`scores, shifts = shift_scores(...)`), a parameter can have a default (`top=0`), and arguments can be named (`build_forecaster(height=64, width=64)`). The first comment inside each of our functions says what it does.

1.7 Libraries

`import numpy as np` loads a library under a short nickname, and `from keras import layers` takes one part.

Table 1. Libraries used in the notebooks.

Library	What it is for
os, glob, shutil, json, time	Paths and folders, finding files, disk space, small settings files, timing.
numpy (np)	Grids of numbers (arrays), used for almost every calculation on maps.
pandas (pd)	Tables with named columns.
matplotlib (plt)	Pictures and charts.
scipy	Only `uniform_filter`, which averages each square with its neighbours.
rasterio	Reads map files (`.tif`) into arrays.
huggingface_hub	Downloads the dataset from Hugging Face.
google.colab	Connects Google Drive and reads Colab secrets.
sklearn	Only the confusion matrix tools.
tensorflow (tf), keras	Build, train and run the neural networks (Keras sits on TensorFlow).
gc	`gc.collect()` frees unused memory.

1.8 Files and folders

A **path** is a file's address as text. `os.path.join(a, b)` glues pieces, `os.makedirs(path, exist_ok=True)` makes a folder (no error if it exists), `os.path.basename` (file name only).`glob.glob(folder + "/*.tif")` finds all `.tif` files (star means anything). `"1800.tif".replace(".tif","")` gives `"1800"` and `int(...)` makes it the number 1800. `with open(path) as file:` opens and closes a file automatically, `json.load` and `json.dump` read and write a dictionary. `np.save` and `np.load` store an array, and `mmap_mode="r"` reads pieces only when needed (files are gigabytes). File types: `.tif` map, `.npy` saved array, `.json` settings, `.csv` table, `.h5` network weights.

1.9 Errors, tries and warnings

`try:` ... `except Exception as error:` means "try this, and if it breaks do that instead of crashing". The download cell loops `for attempt in range(1, 6)` with `try` around the download and `break` on success, so a bad connection restarts up to 5 times. Red text under a cell is usually a harmless warning.

1.10 Classes

`class ClassCapsules(layers.Layer):` is a blueprint for a new kind of network layer. `__init__` stores its settings when it is created, `build` creates the weights it will learn, `call` is the maths when data flows through, and `self` means this very layer.

1.11 Numpy

An **array** is a grid of numbers. A map is 2-D (rows, columns), all maps of a flood are 3-D (time, rows, columns), and **shape** gives the size of each direction.

```
x = np.array([[0.0, 0.2, 0.5], [1.0, 0.0, 0.4]])
x.shape          # (2, 3)
x.mean(axis=0)   # [0.5, 0.1, 0.45]  one average per column
x.mean(axis=1)   # [0.233, 0.467]    one average per row
```

- **Cut a tile:** `depth[step, top:top+64, left:left+64]` is map `step`, 64 rows by 64 columns.
- **Types:** `.astype("float32")` (4 bytes), `float16` (2 bytes, less exact), `uint8` (0 to 255), `bool`.
- **Make and join:** `np.zeros`, `np.full(shape, np.nan)`, `np.arange(0,10,2)`, `np.linspace(0,480,6)`. `np.stack([a,b], axis=-1)` stacks maps into channels, which makes depth, rain and height one 3-layer input. `np.concatenate` joins end to end.
- **New direction:** `x[None]` adds a size-1 direction in front, so shape `(4,64,64,3)` becomes `(1,4,64,64,3)`, a batch of one.
- **Masks:** `depth > 0.3` is a grid of True and False, and `np.mean(depth > 0.3)` is the share of True (True counts as 1). `depth[depth < 0.01] = np.nan` changes only the True squares.
- **Missing values:** `np.isnan`, and `np.nanmean`, `np.nanmax` ignore nan (plain `np.mean` would give nan).
- **Others:** `np.diff` (steps between neighbours), `np.argmax` (position of the biggest), `np.argsort`, `np.maximum(a,0)` (negatives to 0), `np.clip(x,0,10)`, `np.percentile(a,98)`, `np.corrcoef(a,b)[0,1]` (how two lists move together, -1 to 1), `np.where`.
- `np.digitize(value, [0.05,0.15,0.30])` counts how many limits the value reached: 0.04 gives 0, 0.20 gives 2. This turns a flooded share into a severity class (0 None, 1 Minor, 2 Moderate, 3 Severe).

1.12 Pandas

A **DataFrame** is a table. The notebooks collect one dictionary per row in a list and call `pd.DataFrame(rows).display(table)` shows it neatly, `table["Maps"].sum()` adds a column, `table.iloc[0]` is row 0, and `table.to_csv("maps.csv", index=False)` saves it.

1.13 Matplotlib

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))      # a sheet with 3 picture slots
picture = axes[0].imshow(ground_height, cmap="terrain")  # numbers drawn as colours
fig.colorbar(picture, ax=axes[0], label="metres")    # the colour scale
plt.show()
```

`fig` is the sheet and `axes` are the slots. `cmap` is the colour scheme (`Blues` water, `terrain` height, `RdBu` changes, `PuOr` errors, `inferno` heat maps). `vmin` and `vmax` fix the colour range so pictures compare, and `alpha=0.6` makes a picture see-through. The other plot lines only draw bars, lines, dots, titles, the key, or save the file.

1.14 Keras

- `keras.Input(shape=(4,64,64,3))`: 4 maps, 64 by 64, 3 channels. `layers.Conv2D(16, 3, padding="same", activation="relu")`: 16 filters of 3×3, a zero border keeps the size, ReLU turns negatives to 0 (the main guide).

`layers.TimeDistributed(layer)` applies a layer to each of the 4 maps separately. `layers.ConvLSTM2D(32, 3, padding="same")` is the memory layer (the main guide).

- `keras.Model(inputs, outputs)` joins the pieces and `model.summary()` lists layers, shapes and parameters.
- `model.compile(optimizer=keras.optimizers.Adam(0.001), loss="mse")` sets the learning method and the error measure (the main guide). `model.fit(x, y, validation_data=..., epochs=20, batch_size=16, callbacks=[...])` trains: `epochs` are passes through the tiles, `batch_size` is tiles per weight nudge, `validation_data` is the mock test, and `EarlyStopping(patience=4, restore_best_weights=True)` stops after 4 passes with no improvement and returns to the best weights.
- `model.predict(x)` runs the network, `save_weights` and `load_weights` store what it learned, `encoder.trainable = False` freezes a part, `set_random_seed(42)` makes runs repeatable, and `tf.GradientTape()` records the maths so `tape.gradient(a, b)` gives the slope (used by Grad-CAM, the main guide).

2 Notebook 1: Data exploration

2.1 The big picture

The notebook downloads the whole FloodCastBench dataset from Hugging Face, checks that no file is missing, and then looks at the data with tables, pictures and graphs. It also tests which rain map goes with which depth map and tests an old "severe" label. There is no model, nothing is saved to Google Drive, and the files vanish when the Colab session ends. Cell numbers count every cell from 0 (Colab does not show them). The next notebook (Chapter 3) builds the model arrays from what is found here.

A **depth map** is a grid of water depths in metres and a **rain map** a grid of rain in mm per hour, both in `.tif` files. A **dataset** is one region at one square size: Australia 30m, UK 30m, Australia 60m, UK 60m, Pakistan 480m, Mozambique 480m.

2.2 Cells 2 to 5: Setup and settings

Cell 2 runs `!pip install -q rasterio` (`!` means a computer command, `-q` means quiet, and `rasterio` opens `.tif` files). It is needed every session because Colab starts on a fresh machine. Cell 3 loads the toolboxes (`os`, `glob`, `time`, `shutil`, `numpy`, `pandas`, `matplotlib`, `rasterio`, and `snapshot_download` and `HfApi` from `huggingface_hub`). Cell 5 writes every fixed value in CAPITAL LETTERS: the dataset address `REPO_ID`, the temporary folder `DATA_DIR`, the list `REGIONS`, the dictionary `DEPTH_FOLDERS` (dataset name to folder, so a loop can say "for every dataset") and `REGION_OF` (rain is stored per region). None of them prints anything.

```
43
44 # One rain map covers the same time as 6 depth maps
45 DEPTH_MAPS_PER_RAIN_MAP = 6
46
47 # Water above 1 cm counts as wet
48 WET_DEPTH_M = 0.01
```

Depth maps are 300 seconds apart and rain maps 1800 seconds apart, so one rain map covers 6 depth maps. A square is "wet" above 0.01 m (1 cm), a cut-off chosen in the notebook (we think a film thinner than 1 cm is almost dry).

2.3 Cells 7 to 9: Disk space, token and the download

Cell 7 checks the free disk with `shutil.disk_usage` and prints "Free disk space: 62.9 GB" and "Space needed: about 30 GB" (the 30 is typed by hand). Cell 8 reads the Hugging Face token (a secret key) from Colab Secrets inside `try/except`, so a missing key does not stop the notebook. It printed "Token found". Cell 9 is the download:

```
5     try:
6         # Files that are already downloaded are skipped, so a retry carries on where it stopped
7         snapshot_download(repo_id=REPO_ID, repo_type="dataset", local_dir=DATA_DIR,
8                           token=HF_TOKEN, max_workers=8)
9         print("Download complete")
10        break
```

The loop tries up to 5 times (`range(1, 6)`). `snapshot_download` fetches the whole dataset, 8 files at a time (`max_workers=8`), and skips files already present, so a retry carries on where it stopped. On success it prints "Download complete" and `breaks`. On an error the `except` part waits `time.sleep(60)` (60 seconds) and tries again. It prints "Time taken: 60.0 minutes".

Be careful here

The 429 lines. "HTTP Error 429" and "Rate limited. Waiting 113.0s before retry" mean "Too Many Requests". Hugging Face asks the library to slow down and it waits and retries that file (up to 5 times). It is not a failure. The 60.0 minutes covers moving about 27 GB and all those waits. No "Attempt failed" line is visible in the saved output.

Q. Why 5 tries?
A. An hour-long download of 14,870 files can break, and files already fetched are skipped on a retry.

2.4 Cell 10: Check that every file arrived

```
6 missing_files = []
7 for file_name in all_files:
8     if not os.path.exists(os.path.join(DATA_DIR, file_name)):
9         missing_files.append(file_name)
10
11 print("Files in the dataset:", len(all_files))
12 print("Files missing:       ", len(missing_files))
```

`api.list_repo_files` gives the name of every file on Hugging Face. The loop checks each with `os.path.exists` and appends only those that are absent. Output: "Files in the dataset: 14870" and "Files missing: 0". The cell checks that files exist, not that they are undamaged. (Our own count later finds 13,254 depth maps, 1,587 rain maps and 12 ground files, which is 14,853. We do not know what the other 17 are.)

2.5 Cell 12: Four helper functions

A **function** is a named recipe: `def name(inputs):` defines it and nothing runs until it is called. `get_seconds` turns `300.tif` into the number 300 (needed because sorting names as text puts `1200.tif` before `300.tif`). `get_depth_files` lists a dataset's depth files with `glob` and sorts them by that number. `get_rain_files` does the same for a region and its alphabetical sort is enough because names like `20151204-S120000.tif` start with the date. `read_map` opens a map and returns it as a 2D array:

- With `max_size` it reads a shrunk copy (shrink = longer side divided by `max_size`, rounded up). Pictures and rain averages use shrunk maps, depth measurements use full size. We believe rasterio picks squares without averaging, so shrunk numbers are close, not exact.
- `astype("float32")` allows NaN. The files mark "no data" with a huge negative number (the cell comment) and `data < -1e30` turns those into NaN, which `np.nanmean` and `np.nanmax` skip. Cell 20 finds none, so this is only a safety net.

Q. What is NaN and why use it?
A. It means "no value", and it keeps the empty-place marker out of averages and maxima.

2.6 Cell 14: What is in the dataset

For each dataset the cell counts files, converts the time span to hours, reads the map size from the first file, and tests `np.all(np.diff(seconds) == 300)`, that every gap is exactly 5 minutes. A second table does the rain regions (hours are `(len(files) - 1) * 1800 / 3600`).

Depth dataset	Maps	Hours	Size (rows x cols)	5 min?
Australia 30m	2881	240.0	1073 x 1073	True
UK 30m	865	72.0	170 x 275	True
Australia 60m	2881	240.0	536 x 536	True
UK 60m	865	72.0	85 x 137	True
Pakistan 480m	4033	336.0	810 x 441	True
Mozambique 480m	1729	144.0	151 x 138	True

Rain region	Maps	Hours	Size
Australia	481	240.0	1073 x 1073
UK	145	72.0	180 x 285
Pakistan	672	335.5	13035 x 7298
Mozambique	289	144.0	2624 x 2321

All six are exactly 5 minutes apart, so no depth map is missing inside a flood. 240 hours is 2,880 gaps and needs 2,881 maps because both ends count. **Pakistan oddity:** 4,033 depth maps would need 673 rain maps but there are 672, so the last depth map has no rain map and cell 20 stores NaN for it. Rain and depth maps also differ in size (Pakistan depth 810 by 441, rain 13,035 by 7,298, which the report says is 30 m squares), so the next notebook puts them on one grid. The warning "NotGeoreferencedWarning: Dataset has no geotransform" only says the files are plain grids with no map position. It is harmless.

2.7 Cell 16: The four regions

For each region the cell draws ground height (a DEM, digital elevation model), land cover and the water on the ground at the start. `LAND_USE` maps each land-cover code to a name and colour. Squares with under 1 cm of water are set to NaN so the water stands out. The rest only draws (`subplots`, `imshow`, `colorbar`).

Say this: "The four regions differ in height, ground cover and starting water, which is why the floods look different. The model uses ground height as an input." Australia reaches about 210 m with a river and sea, the UK is flat (about 5 to 63 m) with many buildings and up to about 2.4 m of start water, Pakistan is tall (to about 1,500 m on the left) with one 11 m deep spot, and Mozambique (to about 185 m) starts completely dry. The "grey = dry" in the titles shows as white because `axis("off")` also removes the grey.

2.8 Cell 18: Six moments of each flood

```
12      all_wet_values = np.concatenate([m[m > WET_DEPTH_M] for m in maps])
13      deepest_shown = np.percentile(all_wet_values, 98)
```

For each dataset it picks 6 evenly spaced maps (`np.linspace`, for Australia 30m hours 0, 48, 96, 144, 192, 240) and draws them on one colour scale. The top of the scale is the 98th percentile of the wet squares and not the maximum, because a few river or sea squares are 10 or 20 times deeper and would turn everything else the same pale blue. Each dataset has its own scale, so colours are comparable only inside one figure.

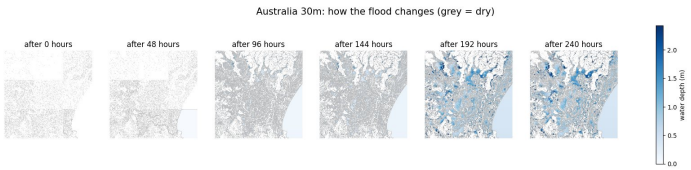


Figure 1. Australia 30m at 0 to 240 hours. Darker blue is deeper (up to about 2.4 m), white is dry.

Say this: "Floods are not instant. Water spreads in thin layers, collects in low places and rivers, and changes slowly, so beating a copy of the last map is hard." Mozambique starts empty and fills slowly, the UK shows its river from the start, and Pakistan covers most of the left and upper part within 67 hours.

2.9 Cells 20 and 21: Rain and water every 30 minutes

Cell 20 is the main, slow cell. For every dataset it keeps one depth map in six (`[::6]`, so 30 minutes apart) and reads it at full size. For each it measures the average depth (`np.nanmean`), the deepest square (`np.nanmax`), the wet share (mean of `depth > WET_DEPTH_M`) and the average rain from the matching rain map, shrunk (NaN if none, only for Pakistan's last step). The data check:

```
21          impossible_values += int(np.isnan(depth).sum())
22          impossible_values += int((depth < 0).sum())
```

It counts missing squares and negative depths. Every dataset printed "impossible values found: 0" over 2,214 maps (481, 145, 481, 145, 673, 289), so there is no missing square and no negative depth in the maps checked (one map in six). Cell 21 draws three graphs per dataset on one hours

axis: rain bars, average depth, and wet-area share.

Say this: “The flood follows the rain. Water rises while it rains and then stays, and the wet area is largest around the heaviest rain.” For Australia 30m the rain peaks near 15 mm per hour around hour 171, the average depth levels off near 36 cm and the wet area peaks at 56.3% at hour 171.5. UK 30m peaks at 49.3% at hour 24.5, Pakistan’s wet area jumps from about 4% to 80% in 15 hours and peaks at 86.1%, and Mozambique starts dry and peaks at 80.3% at hour 54. After the rain stops the wet area slowly shrinks, which we think is water collecting in low places. The 60 m graphs match the 30 m ones except for a higher wet percentage (67.9% against 56.3%), so do not compare wet-area percentages between scales.

Q. Why is rain shrunk and depth not?
A. Rain maps are huge and only their average is needed, while depth is read at full size so its numbers are exact.

2.10 Cell 23: Which rain map goes with which depth map

For the water that appears in each half hour (`np.diff` of the average depth, times 1000 for mm), the cell tests three rain maps: the one with the same time stamp, one 30 minutes earlier and one 30 minutes later.

```
    for step in range(len(water_rise)):
        rain_step = step + shift
        if 0 <= rain_step < len(rain) and not np.isnan(rain[rain_step]):
            rise_values.append(water_rise[step])
            rain_values.append(rain[rain_step])

    return np.corrcoef(rise_values, rain_values)[0, 1]
```

The function pairs each half hour `step` with rain map `step + shift` (-1, 0 or $+1$) if it exists and is not NaN, and `np.corrcoef` gives the **correlation** (1 means they rise and fall together, 0 no link). The right graph plots water added against rain, with a dashed line “water added $= 0.5\times$ rain”, because rain is per *hour* and a step is half an hour. If all rain stays on the ground the dots fall on that line.

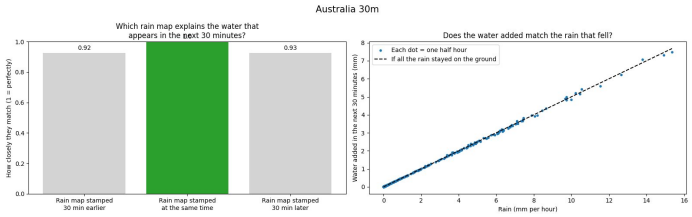


Figure 2. Australia 30m. Left: correlation for the three rain maps (green is the best). Right: each dot is one half hour, rain on x and water added on y.

Say this: “The water that appears in a half hour follows the rain map with the same time stamp, and it equals half the hourly rain rate.” In all six datasets the same-stamp bar is the green one (about 1.0) and the neighbours are 0.92 to 0.97, because rain changes slowly. The “1.0” label sits on the title, a drawing flaw. The report measured 0.494 to 0.502 for the slope. Since rain file j is the rain after depth map j, a forecast must not use it, so the models use only rain that has already fallen and accept the lower 0.92 to 0.97 match (the main guide, 4).

Q. Why do the dots lie on 0.5 times rain?
A. Rain is mm per hour and a step is half an hour, so almost all rain stays on the ground in these simulations.

2.11 Cell 25: Is the old “severe” label measuring the flood?

At the wettest moment of each flood the cell cuts the map into 128 by 128 tiles (`depth[top:top + 128, left:left + 128]`). It applies the old rule and also measures the share of each tile under more than 30 cm:

```
            if np.nanmax(tile) >= 2.0:
                tiles_called_severe += 1
```

```
            # Share of the tile really under more than 30 cm of water
            flooded_shares.append(np.nanmean(tile > 0.3))
```

Dataset	Tiles	Called severe (%)	Actually flooded (%)
Australia 30m	64	93.75	35.73
UK 30m	2	100.00	11.01
Australia 60m	16	100.00	37.13
UK 60m	0	0.00	0.00
Pakistan 480m	18	100.00	33.20
Mozambique 480m	1	100.00	13.87

Say this: “The old label called almost every tile severe because it looked at the deepest single square, while only about a third of a tile was really flooded, so we replaced it.” UK 60m shows 0.00 only because its map has 85 rows, fewer than 128, so no tile fits. UK 30m (2 tiles) and Mozambique (1) rest on very few tiles. The final label (share of normally dry land under 30 cm, 64 by 64 tiles) is in the main guide.

2.12 Cell 27: The summary table

For each dataset it takes the wettest step and shows the deepest square over all measured moments, plus the wet share, the hour and the average depth at that moment.

Dataset	Deepest (m)	Wet area (%)	Hour	Avg depth (cm)
Australia 30m	19.21	56.33	171.5	30.83
UK 30m	6.11	49.33	24.5	10.73
Australia 60m	15.01	67.85	171.5	30.85
UK 60m	4.88	55.56	24.5	10.78
Pakistan 480m	26.67	86.08	132.5	34.53
Mozambique 480m	12.41	80.26	54.0	15.38

The 60 m rows have a lower deepest point and higher wet percentage because the squares are coarser. Pakistan has the largest wet area but its squares are 480 m, so it is not a fair ranking.

Q. Why this dataset, and how do you know the data is complete?
A. FloodCastBench has simulated floods in four regions at several resolutions with matching rain, and we checked 14,870 of 14,870 files present and no impossible values.

2.13 This notebook in ten lines

1. It installs rasterio, loads its libraries and writes every setting in one cell.
2. It downloads 14,870 files (about 27 GB) in about 60 minutes, and the 429 messages are only the website asking it to slow down.
3. It checks that all 14,870 files arrived (zero missing).
4. It defines four helpers: time from a file name, list depth files, list rain files, read a map with NaN for missing values.
5. Its tables show six depth datasets 5 minutes apart and four rain sets 30 minutes apart.
6. It draws ground height, land cover and start water of the four regions.
7. It draws six moments of each flood on one colour scale.
8. It measures depth, wet area and rain every 30 minutes and finds no impossible values in 2,214 maps.
9. It shows that water added follows the same-stamp rain map (about 1.00, neighbours 0.92 to 0.97) and equals half the hourly rain.
10. It shows that the old “severe” label marks almost every tile because of one deep spot, and ends with a summary table.

3 Notebook 2: Data preparation

3.1 The big picture

The raw FloodCastBench files are map pictures of different sizes that lie in different places. The network needs, for each flood, a few tidy blocks of numbers where position (i, j) is the same patch of ground in every block. This notebook builds those blocks and saves them as `.npy` files (Section 1.8). The model notebook (Chapter 4) only loads them. Nothing is learned here.

It downloads 3,822 of the 14,870 files of `mahajananhad/floodcastbench` (depth, rain, ground height (DEM), land cover and position files) in 9.1 minutes, and saves 5.69 GB. Six *datasets* (one flood at one resolution) come out. The project models use four of them (the main guide), the 60 m ones were prepared for the laptop study.

Dataset	Square	Maps	Map size (rows × columns)
Australia_30m	30 m	481	1073 × 1073
UK_30m	30 m	145	170 × 275
Australia_60m	60 m	481	536 × 536
UK_60m	60 m	145	85 × 137
Pakistan_480m	480 m	673	810 × 441
Mozambique_480m	480 m	289	151 × 138

Every folder gets the same files. `depth.npy` (float32, metres) and `rain.npy` (float16, mm per hour) are 3-D: (maps, rows, columns), one map per 30 minutes. `terrain.npy` (float32, metres), `land_cover.npy` (uint8 code) and `permanent_water.npy` (True/False, river or lake) are 2-D: (rows, columns), because the ground does not change in a flood. `map_info.json` holds size, position and projection. All arrays have the rows and columns of the depth map, so square (i, j) is the same ground everywhere. Getting this right is the whole job.

Q. What does this notebook produce?
A. For each of six floods, five aligned `.npy` arrays (depth, rain, terrain, land cover, permanent water) on the depth map’s grid, plus a `map_info.json`.

3.2 Section 0: Setup

Boilerplate: `!pip install -q rasterio` installs the library that opens `.tif` map files (Section 1.1), and the imports bring in file tools, `numpy`, `pandas`, `matplotlib`, `uniform_filter`, the `rasterio` tools `Resampling` (rules for shrinking a map) and `Window` (a rectangle cut from a big map) and the Hugging Face tools. Nothing is printed.

Settings (capital letters mean fixed values, Section 1.2). `STEP_SECONDS = 1800` is the 30 minutes between maps. `WET_DEPTH_M = 0.01` greys out dry squares in the final figures and `FLOOD_DEPTH_M = 0.30` finds the wettest moment. `RAW_DIR`, `SAVE_DIR` and `DRIVE_PREPARED_DIR` are the raw, saved and Drive folders. The token (the `HF_TOKEN` secret) is read in a `try` block and not typed into the notebook, so nobody who gets the notebook sees it.

The `DATASETS` dictionary has six entries, each with `region`, `folder`, `resolution_m`, `scale`, `row_offset` and `col_offset`.

In plain words

Rain, ground height and land cover are all drawn on one fine grid of 30 m squares. A depth map is a sheet laid on that grid. **scale** is the number of 30 m squares along one depth square: 1, 2 or 16 (resolution divided by 30). At scale 16 one depth square covers 256 fine squares. **row_offset** and **col_offset** are how many 30 m squares down and right the depth map's top-left corner lies from the corner of the 30 m files, always counted in 30 m squares.

The offsets are 0 and 0 (Australia), 5 and 5 (UK), 42 and 178 (Pakistan) and 150 and 50 (Mozambique). The notebook does not say how they were found, so do not claim an earlier search. They are tested in the alignment check.

3.3 Section 1: Download only the files we need

`api.list_repo_files(...)` lists all file names without downloading. A loop with four `startswith` tests keeps everything under the Rainfall, DEM, land cover and Georeferenced folders. Depth maps are named by their time in seconds (for example `1800.tif`), and `seconds % STEP_SECONDS == 0` keeps only those on a 30-minute mark (the remainder after dividing by 1800 is 0). Output: `Files in the dataset: 14,870`, `Files we need: 3,822`, of which 2,214 are depth maps (481 + 145 + 481 + 145 + 673 + 289) and 1,608 are rain, ground, land cover and position files.

The download cell tries up to 5 times (`range(1, 6)`). `snapshot_download` with `allow_patterns=files_needed` fetches only the needed files, `max_workers=8` fetches 8 at once, `break` leaves the loop on success and `except` waits 60 seconds and tries again. No failed attempt is printed, so the first try worked. A check then prints `Files missing: 0`.

3.4 Helper functions

Nothing is printed. `get_depth_files` lists and sorts the depth maps by time with `key=get_seconds`, because as text `10800.tif` would sort before `1800.tif`. `get_rain_files`, `dem_path` and `land_cover_path` list or build paths by region.

`read_on_depth_grid` is the key helper. It reads one 30 m map, only the rectangle under the depth map (`Window` at the offsets, width `columns * scale`, height `rows * scale`), and shrinks it onto the depth grid with `out_shape=(rows, columns)` and the rule `resampling=method`. It converts to float32 and stops with `ValueError` if a no-data value (below -10^{30}) appears, which never happened.

Think of it like this

Average versus mode. To turn 16 × 16 small squares into one big one, `average` adds the 256 numbers and divides by 256. That is right for a quantity (height, rain rate). `mode` takes the most common value, like a vote. That is right for categories, where a mean is meaningless: the mean of Water (1) and Buildings (7) is 4, which is Flooded vegetation. At scale 1 nothing is merged.

Q. Why average for terrain and rain but mode for land cover?
A. Height and rain are quantities, so a mean makes sense. Land cover is a category code, so the most common code wins.

3.5 Section 2: Do the layers cover the depth map?

For each dataset the loop reads the depth map size from the file header (`source.height`, `source.width`) into `DEPTH_SHAPES`. The area needed is `row_offset + rows * scale` by `col_offset + columns * scale`, for example 5 + 170 = 175 and 5 + 275 = 280 for UK_30m. For the ground, land cover and first rain file, `fits` is True if the layer is at least that big. A `pandas` table shows 18 rows (6 datasets times 3 layers) and all 18 are True. Australia_30m fits exactly (1073 by 1073). The `NotGeoreferencedWarning` is harmless, only sizes are used here.

3.6 Section 3: Save the depth maps

For each dataset, `np.zeros((len(files), rows, columns), dtype="float32")` makes an empty 3-D block, a loop reads each map with `source.read(1)` into slot `step`, so slot 0 is minute 0 and slot 1 is minute 30. A check warns if the time list differs from 0, 1800, 3600 and so on, and no warning appears. `np.save` writes `depth.npy`. `depth.mean(axis=0)` averages over time and keeps each square's average depth for the alignment check, and `del depth` frees memory. Australia_30m is 481 × 1073 × 1073 numbers at 4 bytes, about 2.2 GB. The six saved sizes match the table above. The counts are flood length in half hours plus one: 10 days (Australia), 3 (UK), 14 (Pakistan) and 6 (Mozambique).

3.7 Section 4: Alignment check

In plain words

The idea. The offsets say where each depth map lies on the 30 m grid, but we do not only trust them. We ask the flood itself. Water collects in rivers and valleys, so squares that are deep on average should be where the land cover map shows a river (**Test 1**) and where the ground map shows a valley (**Test 2**). We slide the river map and the valley map a little in every direction and measure the match. It should be best at the position we use. Like wiggling tracing paper of a river over a photo of a flood until the lines fit. The two tests use different information, so agreement is unlikely to be chance.

`read_full_map` reads a whole 30 m map and fills no-data squares with the mean so they pull neither way. `rivers_map` is `land_cover == 1` as 1.0 and 0.0 (Test 1). `valleys_map` averages the ground in a block about 10 depth squares wide (`uniform_filter`) and subtracts each square's own height, so valley floors score high and hilltops low (Test 2).

`shift_scores`. A border of 3 depth squares is left out so every move stays inside the reference map. The biggest move is `border * scale` 30 m squares (3, 6 or 48) with step `max(1, scale // 4)` (1, 1, 4), giving 7, 13 or 25 moves per direction. Two loops try every (row move, column move) pair. For each, the matching piece of the 30 m map is cut out, shrunk onto the depth grid (`reshape(..., scale, ..., scale).mean(axis=(1, 3))`, which is `Resampling.average` done by hand), and `np.corrcoef` compares it with the average depth map. Correlation is a number from −1 to 1 that says whether two maps go up and down together. Results go into a table `scores`, and out-of-range pieces are skipped with `continue`.

For each dataset one figure has two panels, one per test. `np.nanargmax` and `np.unravel_index` find the best (down, right) move. `imshow` paints the score table (dark purple low, yellow high), with a **red star** at move (0, 0), the position we use, and a **black cross** at the best move.

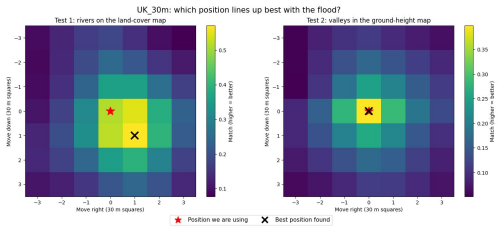


Figure 3. Alignment check for UK_30m. Axes: move right and move down, in 30 m squares. Colour: correlation, purple low and yellow high. Left: Test 1 (rivers). Right: Test 2 (valleys).

Say this: in each panel the match is a bright blob centred at or right next to the red star, falling away smoothly, so the assumed offsets are about right. In Test 1 (left) the cross is one square right and one down of the star but the neighbouring cells are almost as bright. In Test 2 (right) star and cross share a cell.

Decision rule. A move below half a depth square makes no real difference. `rivers_close` and `valleys_close` are True when the larger of `abs(down)` and `abs(right)` is at most `scale / 2` (0.5, 1, 8). `tests_agree` is True if the two best moves are within one `shift_step`. Both close gives "Keep the position". Otherwise, if the tests agree, "Both tests say move it". Otherwise "Tests disagree, look closer". Best moves are (down, right) in 30 m squares.

Dataset	Test 1 (rivers)	Test 2 (valleys)	Decision
Australia_30m	(0, 1)	(0, 0)	Both tests say move it
UK_30m	(1, 1)	(0, 0)	Both tests say move it
Australia_60m	(1, 1)	(0, 0)	Keep the position
UK_60m	(1, 1)	(0, 0)	Keep the position
Pakistan_480m	(0, 0)	(0, 4)	Keep the position
Mozambique_480m	(0, 8)	(0, 0)	Keep the position

At 60 m and 480 m the best moves are at most half a depth square, so they count as close. At 30 m the smallest possible move (30 m) is more than half a square (15 m), so only an exact 0 counts as close.

Be careful here

"Both tests say move it" sounds stronger than the numbers. In both 30 m rows Test 2 says (0, 0), which means keep. Only Test 1 asks for a move, by one 30 m square, the smallest move that exists. Read it as "the tests agree within one square, and one prefers a one-square shift". The report says: "Two independent checks agreed within about one cell." Nothing is moved. Later cells read the offsets from `DATASETS`. We think the authors judged one square small enough to ignore, but the notebook does not say so.

Q. What is the alignment check for?
A. It tests that each depth map really sits where we assume on the 30 m grid, by sliding river and valley maps and seeing where they best match the average water depth.

Q. Two datasets say "Both tests say move it". Did you move them?
A. No. The best move is one 30 m square, the smallest possible, and Test 2 puts the best position at exactly zero, so the tests agree within about one cell and we kept the offsets.

3.8 Section 5: Ground height, land cover and rain

For each dataset, `read_on_depth_grid` with `average` gives `terrain.npy` of shape (rows, columns). Land cover uses `mode` and `.astype("uint8")` (a whole number from 0 to 255, enough for codes up to 11, 1 byte per square). `permanent_water = land_cover == 1` is True for rivers and lakes.

Rain timing. Rain file j is the rain in the 30 minutes after depth map j: `rain[0]` lies between depth maps 0 and 1, `rain[1]` between 1 and 2. The text cell says so, and the report (Section 3.2) says it was measured: the files are mm per hour, so half an hour should raise the average depth by 0.5 times the file value, and on the active steps the measured rise was 0.494 to 0.502 times the file value in all four floods. This notebook does not repeat that test, it only stores the maps in this order. The code makes `np.zeros(..., dtype="float16")`, and for each `step` with a rain file (`step < len(rain_files)`) reads it with `average` into slot `step`. Otherwise the slot stays 0 and `missing_rain` goes up by one. **Pakistan_480m reports 1 missing**, so its last rain map (number 672) is all zeros. We think it has 673 depth maps but 672 rain files, but the notebook does not say so. The other five report 0.

In plain words

Why float16. 2 bytes instead of 4, so every `rain.npy` is exactly half of the `depth.npy` of the same dataset (1107.6 MB against 2215.2 MB for Australia_30m). The comment says rain stays below about 20 mm per hour, where float16 is accurate to about 0.02 mm per hour.

Q. What does `rain[t]` mean?
A. It is the rain in mm per hour that falls in the half hour after depth map t, between map t and map $t+1$.

3.9 Section 6: Map details

A world file (`.tfw`) is a tiny text file with 6 numbers: square width (30), two rotation terms (0), square height (negative, rows go down), and the x and y of the *centre* of the top-left square in the region's projection. The code moves half a square left and up to get the outer corner, then adds

`col_offset` and `row_offset` squares to get the depth map's corner. This is where the offsets become a real position. `map_info.json` stores the name, region, size, rows, columns, number of maps, 1800 seconds, the EPSG code (a number naming the projection), a rain note and a six-number `"transform"` that turns row and column into a position on Earth. The check table shows that for all six datasets the world file's square size times the scale equals the expected size (30, 30, 60, 60, 480, 480 m) and `Map rotated?` is False, so the maps are plain north-up grids.

3.10 Section 7: Final check on the saved data

`np.load(path, mmap_mode="r")` opens the big arrays as a read-only window on the disk, so only the maps asked for are read (`depth.npy` of Australia is 2.2 GB). For each flood the code finds the share of squares deeper than 30 cm for every map, takes the biggest (`np.argmax`) as the wettest moment, and draws four panels: depth (squares under 1 cm set to `nan` and drawn grey, colour scale ended at the 98th percentile so the deepest 2 percent do not stretch it, red contour at the permanent water), ground height, land cover and `rain[wettest]`. The wettest moments are Australia_30m 181 h, UK_30m 41, Australia_60m 181, UK_60m 40, Pakistan_480m 180 and Mozambique_480m 108.

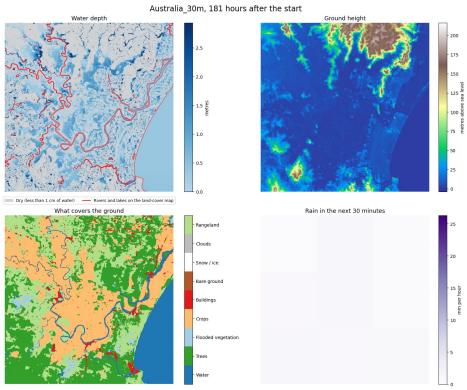


Figure 4. Australia_30m at 181 hours. Top left: depth, light blue shallow to dark blue deep, grey dry, red line permanent water. Top right: ground height. Bottom left: land cover. Bottom right: rain in the next 30 minutes.

Say this: the deepest water (up to about 2.9 m) lies in the river channels and the red outlines sit on the dark blue channels, so depth, ground and land cover line up. The rain panel is almost white because there is almost no rain at that moment.

The last cell lists all files with `os.listdir` and `os.path.getsize` (36 rows, 6 files in 6 folders) and prints `Total saved: 5.69 GB`. `depth.npy` is the biggest, every `rain.npy` is half of its depth file, and `terrain.npy` is four times `land_cover.npy` (4 bytes against 1 per square, for example 4.6 MB against 1.2 MB for Australia_30m). Pakistan_480m is depth 961.6 MB and rain 480.8 MB.

Q. What does `mmap_mode="r"` do?
A. It opens a big `.npy` file without loading it all into memory. Only the parts the code asks for are read, and the file is read-only.

3.11 Section 8: Compare with the copy on Google Drive

`drive.mount` connects Drive (output `Mounted at /content/drive`). `same_array` opens both files with `mmap_mode="r"`, returns `False` if shape or type differ, compares 2-D arrays in one go with `np.array_equal(..., equal_nan=True)` and 3-D arrays one map at a time to save memory. Two loops cover 6 datasets times 5 arrays and mark each `identical`, `DIFFERENT` or `not on Drive`. **All 30 say `identical`.**

In plain words

Each pair has the same shape, type and value in every square of every map. So the data this notebook builds from the raw files is exactly what the model notebook loads from Drive and what the models were trained and tested on, and anyone can rerun this to get the model's input. It does not prove the data are physically right (the alignment check and final pictures are for that), and `map_info.json` is not compared.

Q. Why compare with the Drive copy?
A. The model notebook uses the Drive copy, and this shows the raw files produce exactly that copy, so the data behind the results can be recreated. Drive is only read.

3.12 This notebook in ten lines

1. It installs `rasterio`, imports libraries and writes its settings, including `DATASETS` with scale and offsets.
2. It lists the 14,870 files and keeps the 3,822 it needs, depth maps only on a 30-minute mark.
3. It downloads them in 9.1 minutes and finds 0 missing.
4. It checks that every layer covers the depth map's rectangle: all 18 checks True.
5. It saves each flood as one block `depth.npy`, for example 481 maps of 1073 by 1073 for Australia at 30 m.
6. It slides river and valley maps and finds the best match with the average depth. Both tests agree within one cell of the assumed place, and the offsets are kept.
7. It cuts ground height (average), land cover (mode) and rain (average, float16) down to the depth grid. Rain file *j* is the rain after depth map *j*.
8. Pakistan has its last rain map missing, left as zero.
9. It saves size, projection and position in `map_info.json`, loads the data back and draws the wettest moment of each flood to see that all layers line up.
10. It lists 5.69 GB of files and finds all 30 arrays identical to the Drive copy.

4 Notebook 3: The flood model

Big picture. This notebook cuts the prepared flood arrays (Chapter 3) into tiles, builds Model A (CNN-LSTM), loads its saved weights, tests it on whole maps and compares it with simple forecasts. Then it builds Model B, which starts from the trend forecast. It reads four prepared floods and six saved models from Drive and writes figures and tables to `model_outputs`. Nothing is trained again (`TRAIN_NEW` and `TRAIN_V2` are `False`). The title cell asks for a T4 GPU. Parts 5 to 8 (capsule network, Grad-CAM, rules, summary) are in the second half.

4.1 Section 0 and settings

Setup cells. `import` brings in toolboxes with nicknames: `np` (arrays), `pd` (tables), `plt` (figures), `tf` and `keras` with `layers` (networks), `uniform_filter` (neighbour average), `drive`. The GPU cell prints a list with one `PhysicalDevice(... 'GPU')` (`[]` would mean none). `drive.mount(...)` prints `Mounted at /content/drive`, then four folders are built from `PROJECT_DIR` and `os.makedirs(..., exist_ok=True)` creates missing output folders.

Settings cell. Every number is written once here (names in capitals are constants).

```
# Two models: each one learns on one flood and is tested on another flood it has never seen
PAIRS = [
    {"name": "30m", "train": "Australia_30m", "test": "UK_30m"},
    {"name": "480m", "train": "Pakistan_480m", "test": "Mozambique_480m"},
]
```

`PAIRS` is a list of two dictionaries. The **30 m model** learns on `Australia_30m` and is tested on `UK_30m`. The **480 m model** learns on `Pakistan_480m` and is tested on `Mozambique_480m`. Two networks, because 30 m and 480 m squares behave differently (the main guide). `REGION_NAMES` gives table names such as `"UK (30 m)"` and fixes the order.

Constant	Value	Meaning and reason
`STEP_HOURS`	0.5	Maps are 30 minutes apart. Map number × 0.5 is hours.
`WET_DEPTH_M`, `FLOOD_DEPTH_M`	0.01, 0.30	Wet: deeper than 1 cm (the water rule takes water away only there). Flooded: deeper than 30 cm (CSI, severity). No reasons given.
`SEVERITY_LIMITS`	0.05, 0.15, 0.30	Cut the flooded share of dry land into None, Minor, Moderate, Severe (capsule network, later).
`TRAIN_SHARE`, `VALIDATION_SHARE`	0.70, 0.15	Split by *time*: first 70% train, next 15% check (validation), last 15% test. Australia (481 maps): training ends at map 336, checking at 408, testing 408 to 480.
`INPUT_STEPS`, `CHANNELS`	4, 3	4 maps (1.5 hours), each with 3 layers: depth, rain, local ground height. No reason stated for 4 (general reasoning: enough to see a trend).
`TILE_SIZE`	64	Training pieces are 64 by 64, because whole Australian maps (over a million squares) do not fit GPU memory in large numbers. The network only slides windows, so it works on whole maps too. 64 has no stated reason.
`TRAIN_TILES`, `VALIDATION_TILES`, `TEST_TILES`	4000, 600, 1000	Random tiles for training, checking and (capsule network) testing. No reason given.
`CHANGE_LIMIT_M`, `CALM_TILE_SHARE`	0.01, 1.0	A tile "changes" if some square moved more than 1 cm in 30 minutes. `CALM_TILE_SHARE` is the share of calm tiles kept. At 1.0 nothing is skipped, so the filter is off.
`RAIN_SCALE`	10.0	Rain (mm per hour) divided by 10 to match the scale of the other inputs (comment).
`TERRAIN_WINDOW`	15	Height is compared with the average of the 15 by 15 squares around it. No reason stated.
`EPOCHS`, `BATCH_SIZE`	20, 16	At most 20 passes through the training tiles. Weights are nudged after every 16 tiles, so 4000 tiles give 250 updates per pass.
`LEARNING_RATE`	0.001	Size of each nudge, the usual Adam default (general reasoning). The report lists "Adam, 10^{-3}, constant".
`TRAIN_NEW`, `TRAIN_V2`	False	Switches for Model A and B. False loads saved weights.

`keras.utils.set_random_seed(42)` fixes Python, numpy and TensorFlow random numbers, so tile positions and starting weights repeat in every run (42 has no meaning). The last lines build the blue colour scale and draw `nan` squares grey. The cell prints nothing.

Q. Why a time split and not a random split?
A. A forecast is about the future, and random mixing would put training maps between test maps, making the test too easy.

4.2 Helper functions, `load_region` and loading the data

Helpers (definitions only). `save_figure`, `save_table` write `.png`, `.csv`. `show_water` draws squares under 1 cm grey, the rest blue. `cut_into_tiles` loops `range(0, rows - 64 + 1, 64)` over rows and columns and lists the top-left corners, so every tile fits inside the map. The UK (170 by 275) gives $2 \times 4 = 8$ tiles, Mozambique (151 by 138) 4, Pakistan 72, Australia 256.

`load_region(dataset)` loads one flood and returns everything in one dictionary.

Lines 6 and 7: `np.load` reads `depth.npy` and `rain.npy` (maps, rows, columns), and `np.nan_to_num(..., copy=False)` turns `nan` into 0 in place (Australian depth alone is about 2.2 GB). Lines 10 and 11: missing terrain gets the average height (`np.nanmean`). Line 14: `uniform_filter(terrain, size=15)` is the 15 by 15 neighbour average, so `terrain -` average is the **local height**: positive on a hilltop, negative in a dip or river bed, near 0 on a plain, whatever the height above sea. Line 15 divides by the standard deviation (`max(..., 1e-6)` avoids division by zero).

Line 18 loads `permanent_water.npy` (True for rivers and lakes) and line 30 flips it with `~`, so `dry_land` is the normally dry ground. Lines 21 to 32 read `map_info.json` and fill a dictionary with depth, rain, local height, dry land, info and number of maps. Lines 33 and 34: `train_end = int(0.70 * 481) = 336`, `validation_end = int(0.85 * 481) = 408`. Australia has training maps 0 to 335, checking 336 to 407 and test 408 to 480 (73 maps).

Load all four floods. A loop over `REGION_NAMES` calls `load_region` and fills `regions` (takes minutes). It prints the shapes (maps, rows, columns): Australia_30m (481, 1073, 1073), UK_30m (145, 170, 275), Pakistan_480m (673, 810, 441), Mozambique_480m (289, 151, 138), as in Table 2 of the report.

First target and split table. `FIRST_TARGET = INPUT_STEPS + 1 = 5`. To forecast map 5 the model needs maps 1 to 4, and map 1 needs the rain stored at number 0, so the first forecastable map is map 5, hour 2.5. The cell turns the split points into hours and saves `01_data_split`.

Australia: train 2.5 to 168.0 h, validation 168.0 to 204.0, test 204.0 to 240.5. Pakistan: train 2.5 to 235.5, validation 235.5 to 286.0, test 286.0 to 336.5. The UK and Mozambique have dashes for train and validation: they are never used for training or checking, and their whole flood is the test.

4.3 Tiles: `make_input`, `make_training_tiles`

`make_input(region, last_step, top, left, height, width)` builds one input: the four newest maps up to `last_step`, three layers each, for a tile or the whole map. The model then forecasts map `last_step + 1`.

```
5       if height is None:
6           height = region["depth"].shape[1]
7           width = region["depth"].shape[2]
8
9       maps = []
10
11      for step in range(last_step - INPUT_STEPS + 1, last_step + 1):
12          depth = region["depth"][step, top:top + height, left:left + width]
13
14          # rain[step - 1] is the rain that fell before this map, so it has already happened
15          rain = region["rain"][step - 1, top:top + height, left:left + width].astype("float32") / RAIN_SCALE
16
17          height_map = region["local_height"][top:top + height, left:left + width]
18
19          # Stack the 3 layers of this map
20          maps.append(np.stack([depth, rain, height_map], axis=-1))
21
22      # Shape: (4 maps, height, width, 3 layers)
23      return np.stack(maps).astype("float32")
```

Lines 5 to 7: with no `height` the whole map is used, otherwise the tile. Line 11: `range(last_step - 3, last_step + 1)` gives the four map numbers, oldest first. Line 12: `depth[step, top:top + height, left:left + width]` cuts the tile. Line 15: rain from `step - 1`, `float32`, divided by `RAIN_SCALE`. Line 17: local height (no time axis, the ground does not change). Line 20: `np.stack([...], axis=-1)` puts the three maps on top of each other, (64, 64, 3). Line 23: `np.stack(maps)` stacks the four in time order, shape **(4, height, width, 3)**: (4, 64, 64, 3) for a tile and (4, 1073, 1073, 3) for whole Australia.

Be careful here

Rain file *j* holds the rain that falls in the 30 minutes *after* depth map *j*. So `rain[step - 1]` is the 30 minutes just before map `step`. The rain after the newest input map, `rain[last_step]`, is in the future and never given (an earlier "hybrid input" version gave it).

`make_training_tiles(region, first_target, last_target, count)` cuts `count` random 64 by 64 tiles at random times, each with its input and its answer (the true depth 30 minutes after the newest input map).

```
12          target_step = np.random.randint(first_target, last_target)
13          top = np.random.randint(0, rows - TILE_SIZE + 1)
14          left = np.random.randint(0, columns - TILE_SIZE + 1)
15
16          target = region["depth"][target_step, top:top + TILE_SIZE, left:left + TILE_SIZE]
17          last = region["depth"][target_step - 1, top:top + TILE_SIZE, left:left + TILE_SIZE]
18
19          # Skip some calm tiles (with CALM_TILE_SHARE = 1.0 nothing is skipped)
20          water_changes = np.abs(target - last).max() > CHANGE_LIMIT_M
21          if not water_changes and np.random.rand() > CALM_TILE_SHARE:
22              continue
23
24          inputs.append(make_input(region, target_step - 1, top, left, TILE_SIZE, TILE_SIZE))
25          targets.append(target[:, :, None])
26          positions.append((target_step, top, left))
```

Lines 6 to 8 make empty lists and line 10 a `while` loop (a tile may be skipped, then we retry) until `count` tiles exist. Lines 12 to 14: `np.random.randint` picks the target map from `first_target` to `last_target - 1`, and a random corner (`+ 1` because the upper limit is excluded) so the tile always fits. Lines 16 and 17: the true tile at target time and the tile one step earlier. Lines 20 to 22: if no square changed more than `CHANGE_LIMIT_M`, the tile is calm and is skipped when `np.random.rand()` is above `CALM_TILE_SHARE`, which never happens at 1.0. Line 24: `make_input(region, target_step - 1, ...)`. Lines 25 and 26: `target[:, :, None]` gives shape (64, 64, 1), and `positions` remembers (time, top, left) for later. Line 28 returns the arrays (count, 4, 64, 64, 3), (count, 64, 64, 1) and the list.

Making the tiles (only the training flood of each pair is used):

Training tiles: 4000 with targets from `FIRST_TARGET` to `train_end`. Checking tiles: 600 with targets from `train_end` to `validation_end`, used to measure error after each epoch and never to change weights. It stores them in `tiles` and prints `(4000, 4, 64, 64, 3)` and `(600, 4, 64, 64, 3)` for both models.

4.4 Section 2: Model A, `build_forecaster`

Two CNN layers find shapes (water edges, slopes, rain patches) in each of the 4 maps, a ConvLSTM reads them in order and remembers how the water moved, and a decoder turns that memory into the change in depth. Next map = last map plus change, never below 0.

```
4       # Input: 4 maps, each height x width, with 3 layers
5       inputs = keras.Input(shape=(INPUT_STEPS, height, width, CHANNELS), name="past_maps")
6
7       # CNN: the same 2 convolution layers are used on each of the 4 maps
8       x = layers.TimeDistributed(layers.Conv2D(16, 3, padding="same", activation="relu"), name="cnn_1")(inputs)
9       x = layers.TimeDistributed(layers.Conv2D(16, 3, padding="same", activation="relu"), name="cnn_2")(x)
```

Line 5: `keras.Input(shape=(4, height, width, 3))` is the input, with the number of examples left free (`None`). Line 8: `Conv2D(16, 3, padding="same", activation="relu")` has **16 filters**, each a 3 by 3 window sliding over the map. `padding="same"` keeps the size. ReLU sets negatives to 0. `TimeDistributed` applies it to *each of the 4 maps with the same filters*: (4, 64, 64, 3) becomes (4, 64, 64, 16). The notebook gives no reason for 16 (general reasoning: a few small filters detect simple patterns). Line 9: the second CNN layer, on the result. General reasoning (no one-layer version is tested): it combines simple patterns into bigger ones, since

its 3 by 3 window sees 5 by 5 original squares.

```
    # ConvLSTM: reads the 4 maps in order and keeps a memory map
    memory = layers.ConvLSTM2D(32, 3, padding="same", name="memory")(x)

    # Decoder: memory -> predicted change in depth (the last layer starts at zero)
    x = layers.Conv2D(16, 3, padding="same", activation="relu", name="decoder")(memory)
    change = layers.Conv2D(1, 1, kernel_initializer="zeros", name="depth_change")(x)

    # Next map = last depth map + change, and never below 0
    last_depth = inputs[:, -1, :, :, 0:1]
    next_depth = layers.ReLU(name="next_depth")(layers.Add()([last_depth, change]))

    return keras.Model(inputs, next_depth, name="cnn_lstm_forecaster")
```

Line 12: `ConvLSTM2D(32, 3, padding="same")` reads the 4 maps oldest first, updates a memory that is itself a map, and returns only the final memory: (4, 64, 64, 16) to (64, 64, 32). No reason for 32 is given (we think twice the 16 gives room). Line 15: decoder, a 3 by 3 convolution with 16 filters and ReLU mixing the 32 memory maps into 16. Line 16: `depth_change`, `Conv2D(1, 1)`, one 1 by 1 filter that combines the 16 numbers of each square into the predicted change in metres. No activation, so the change may be positive or negative. `kernel_initializer="zeros"` starts its weights at 0, so the first predicted change is 0 everywhere. Line 19: `inputs[:, -1, :, :, 0:1]` is all examples, the last map, all rows and columns, layer 0 (depth). `0:1` keeps the layer axis, shape (64, 64, 1). Line 20: `layers.Add()` adds last depth and change, `ReLU` removes negatives (water cannot be below 0). Line 22 packs it into a `keras.Model`.

Think of it like this

The last layer starts at zero, so before training the output is the last depth. Training starts from "copy the last map" and only learns the *difference*, which is easier because most of the map hardly changes in 30 minutes.

Whole-map copy (lines 25 to 31). `forecaster_for_whole_map` builds Model A at whole-map size and copies the learned numbers with `get_weights()` and `set_weights(...)`. This works because every layer is a sliding window with no flatten or dense layer. We train on tiles (memory) and test on whole maps (a real forecast).

Layer summary (line 35 `summary()`):

Layer (type)	Output shape	Param #	How the number arises
past_maps (InputLayer)	(None, 4, 64, 64, 3)	0	holds the input
cnn_1 (TimeDistributed)	(None, 4, 64, 64, 16)	448	$16 \times (3 \cdot 3 \cdot 3 + 1)$
cnn_2 (TimeDistributed)	(None, 4, 64, 64, 16)	2,320	$16 \times (3 \cdot 3 \cdot 16 + 1)$
memory (ConvLSTM2D)	(None, 64, 64, 32)	55,424	$4 \times 32 \times (3 \cdot 3 \cdot (16 + 32) + 1)$
decoder (Conv2D)	(None, 64, 64, 16)	4,624	$16 \times (3 \cdot 3 \cdot 32 + 1)$
get_item	(None, 64, 64, 1)	0	the cut `inputs[:, -1, :, :, 0:1]`
depth_change (Conv2D)	(None, 64, 64, 1)	17	$16 + 1$
add, next_depth (ReLU)	(None, 64, 64, 1)	0	last depth plus change, no negatives

Totals: `Total params: 62,833 (245.44 KB)`, all trainable. Per filter a convolution has one weight per window square and input layer, plus one bias. The ConvLSTM has four gates, each looking at the 16 input layers and its own 32 memory layers, so 55,424 of the 62,833 numbers sit there. `cnn_1` is only 448 because the four maps share filters.

Q. Why a 1 by 1 filter at the end?
A. It only mixes the 16 numbers of each square into one change value and does not look at neighbours.

4.5 Load (or train) Model A, training curves

```
    forecaster = build_forecaster()
    forecaster.compile(optimizer=keras.optimizers.Adam(LEARNING_RATE), loss="mse")

    if TRAIN_NEW or not os.path.exists(weights_path):
        print("Training Model A", pair["name"])

        # Stop when the checking error has not improved for 4 epochs, and keep the best weights
        stop_early = keras.callbacks.EarlyStopping(patience=4, restore_best_weights=True)

        history = forecaster.fit(
            data["train_inputs"], data["train_targets"],
            validation_data=(data["validation_inputs"], data["validation_targets"]),
            epochs=EPOCHS, batch_size=BATCH_SIZE, callbacks=[stop_early], verbose=2)

        histories[pair["name"]] = history.history
        forecaster.save_weights(weights_path)
    else:
        print("Loading Model A", pair["name"])
        forecaster.load_weights(weights_path)
```

Lines 6 and 7: this model's tiles and weights file (`forecaster_30m.weights.h5` or 480m). Lines 9 and 10: a fresh Model A, `compile` with `Adam(LEARNING_RATE)` (decides each nudge) and `loss="mse"` (squared forecast minus truth, averaged over squares). Line 12: training runs if `TRAIN_NEW` is on or the file is missing, so it is skipped here. Line 16: `EarlyStopping(patience=4, restore_best_weights=True)` watches `val_loss` on the checking tiles, stops when it has not improved for 4 epochs in a row and puts back the best epoch's weights (guard against overfitting, no reason for 4 given). Lines 18 to 21: `fit` is the training: inputs and targets, `validation_data` (measured only), `epochs=20`, `batch_size=16` (250 updates per epoch), the early-stop helper, `verbose=2` (one line per epoch with `loss` and `val_loss`). Line 24 `save_weights`. Lines 25 to 27: otherwise `load_weights`.

It prints `Loading Model A 30m` and `480m` (nothing trained) and a harmless `UserWarning: Skipping variable loading for optimizer 'adam'...`. The saved file also holds Adam's memory, only needed to continue training.

Training curves. The code starts with `if len(histories) > 0:` and `histories` is empty, so the output is empty. A trained model

would draw `100 * np.sqrt(loss)` (RMSE in cm), and a rising checking line with a falling training line means overfitting (for 30 m Model A the checking error was lowest after the second epoch, the main guide).

4.6 Section 3 and 3a: the water-balance rule

Scores. RMSE is the typical error per square (cm). **Skill** $= 1 - (\text{model error}/\text{copy error})$: 0 is copying, 1 is perfect, below 0 is worse than copying (the main guide). **CSI** measures how well flooded areas (above 30 cm) match. **3a**: almost all rain stays on the ground, so the rise of the average depth in 30 minutes (mm) is about the rain (mm per hour) times 0.5 h. The rule fixes only the *total* water, the network decides where it goes, and only rain that has already fallen is used.

The check cell. Per flood: `depth.mean(axis=(1, 2), dtype="float64")` is the average depth of each map, `1000 *` makes millimetres and `np.diff` the rise per step. `rain_now_mm` is average rain times 0.5 h for the same half hour, `rain_before_mm` the same shifted by one step, the only rain a forecast knows. `ax.scatter` draws the dots, `np.corrcoef(a, b)[0, 1]` the correlations.

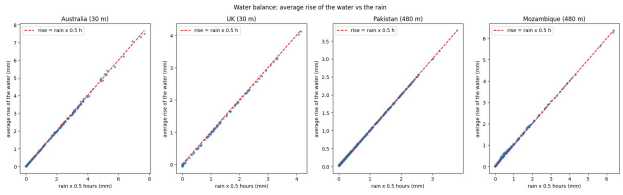


Figure 5. Water balance, one panel per flood. Each dot is one 30-minute step. Across: rain times 0.5 h (mm). Up: rise of the average depth (mm). Red dashed: rise = rain $\times$ 0.5 h.

Say this: in all four floods the dots follow the red line, so rain explains the rise of the total water.

Region	Total rise / total rain	Corr., same half hour	Corr., half hour before
Australia (30 m)	0.999	1.0	0.925
UK (30 m)	0.946	1.0	0.945
Pakistan (480 m)	1.028	1.0	0.969
Mozambique (480 m)	1.045	1.0	0.929

The balance holds within about 5% (the report blames water crossing the map edge, and the Indus inflow in Pakistan). With the same half hour's rain the rise is almost perfect, but a forecast cannot know it. With the half hour before, the correlation is 0.925 to 0.969, so the rule is a good but not perfect guess.

`apply_water_balance(predicted, last, past_rain)` shifts a forecast so its total rise matches the rain.

```
7     allowed_rise = 0.5 * float(np.mean(past_rain)) / 1000
8
9     # Extra water the network added (positive) or rain water it missed (negative)
10    extra_water = (float(np.mean(predicted_change)) - allowed_rise) * predicted.size
11
12    corrected = predicted.copy()
13
14    if extra_water < 0:
15        # Missing rain water: rain falls everywhere, so add it to every square
16        corrected = corrected - extra_water / predicted.size
17    else:
18        # Extra water: take it away only from squares that have water
19        wet = corrected > WET_DEPTH_M
20        if wet.sum() > 0:
21            corrected[wet] = corrected[wet] - extra_water / wet.sum()
22
23    # Depth can't be negative
24    return np.maximum(corrected, 0)
```

Line 4 takes forecast minus last map. Line 7: `allowed_rise = 0.5 * mean(past_rain) / 1000`, in metres per square. Line 10: the average predicted change minus the allowed rise, times `predicted.size`, is `extra_water` of the whole map (positive means too much, negative too little). Lines 12 to 16: if `extra_water < 0`, the missing water is added equally to every square (rain falls everywhere). Lines 17 to 21: otherwise `wet = corrected > WET_DEPTH_M` and the extra water is taken equally from wet squares only. Line 24: `np.maximum(corrected, 0)` removes negative depths, which can add back a little water, so the match is almost always exact but not always.

Example: last map $0, 0, 0.20, 0.40$ m, rain 4 mm per hour, allowed rise 0.002 m. Forecast changes $+0.001, +0.001, +0.003, +0.005$ (average 0.0025): extra 0.002 in total, the two wet squares lose 0.001 each, average change now exactly 0.002.

4.7 Skill, `evaluate_region` and the Model A results

`critical_success_index(predicted, true)` makes two True/False maps (deeper than 30 cm in forecast and in truth) and counts `hits` (both, `&`), `misses` (truth only, `~` is not) and `false_alarms` (forecast only). Result `hits / (hits + misses + false_alarms)`, `nan` if all are 0. Example: 18 hits, 2 misses, 4 false alarms give $18/24 = 0.75$. Squares dry in both are ignored.

`evaluate_region(forecaster, region, first_target, last_target)` forecasts every whole map of a test period.

Line 24: `make_input(region, target_step - 1)` with no tile gives the whole-map input, `[None]` adds a batch axis. Line 25: `forecaster(model_input, training=False)` runs the network and `[0, :, :, 0]` keeps a plain map. Lines 27 and 28: the true map and the previous real map (the copy forecast). Line 31: the newest rain layer times `RAIN_SCALE` (back to mm per hour), and line 32 applies the rule. Lines 34 to 43 store, per forecast, the *sum* of squared errors of the model, the model with the rule and copying, the number of squares and three CSI values. Lines 46 to 49 remember the step with the largest water change for the map figure.

Running the test. For each pair it builds whole-map models for both floods, tests the training flood on maps from `validation_end` to the end (last 15%, never trained on) and the never-seen flood from `FIRST_TARGET` to its end. It prints `30m done`, `480m done`. The numbers of

forecasts are 73, 140, 101 and 284. The typical error is `100 * np.sqrt(error / cells)` in cm over all squares including dry ones, and skill is `1 - model_error / copy_error` on totals.

Region	Tested on	Fcsts	Copy (cm)	A (cm)	A+rule (cm)	Skill A	Skill A+rule
Australia (30 m)	unseen time	73	0.112	0.175	0.147	−1.449	−0.710
UK (30 m)	never-seen	140	0.520	0.433	0.429	0.305	0.320
Pakistan (480 m)	unseen time	101	0.103	0.084	0.080	0.345	0.404
Mozambique (480 m)	never-seen	284	0.548	0.505	0.498	0.150	0.172

CSI, copy / A with rule: Australia 0.9995 / 0.9992, UK 0.9888 / 0.9919, Pakistan 0.9993 / 0.9994, Mozambique 0.9678 / 0.9732. All are near 1 because deep squares stay deep for the next half hour.

Why Australia is −1.449. Skill is about $1-(0.175/0.112)^2 = 1-2.44$, so Model A's squared error is about 2.4 times that of copying. The errors are tiny (about 1.1 mm for copying, 1.75 mm for A) because late in the Australian flood the water barely moves. Negative skill means worse than copying, not a large error. The report notes Model A probably added a little water almost everywhere (not measured further). The rule helps (0.175 to 0.147 cm) but not enough. On the other three tests A beats copying (0.305 means about 30% of copying's squared error removed).

Q. Model A is worse than copying in Australia. Why?
A. Late in that flood the water hardly moves, so copying is almost perfect, and Model A adds a little water almost everywhere. Skill −1.449, about 2.4 times the squared error. Report says not measured further.

4.8 Model A figures

Skill bars. `ax.bar` draws two bars per test (blue A, green A+rule, shifted ∓0.2), `ax.axhline(0, color="black")` marks skill 0 (copying), the rest sets labels and saves `03_skill_per_region`. Australia is far below the line (−1.449 blue, −0.710 green), the others above it, green a little taller. **Say this:** Model A beats copying in three of four tests, fails in late Australia, and the rule helps every time.

Error at every step. Four panels (figure `model_c29_3`, same layout as the later Figure with the trend) plot `100 * np.sqrt(squared error / cells)` per forecast against the hour: grey copy, blue A, green A+rule.

Say this: every forecast is wrong by more when the water changes fastest, and A never gets near zero. Australia: blue stays above grey (about 0.21 to 0.16 cm, copying falls from 0.17 to 0.08). UK: copying peaks near 1.2 cm around hour 25, A about 0.9. Pakistan: A is level with or slightly above copying, but at hour 333 copying peaks at 0.30 and A at 0.15. Mozambique: A is below copying early, the peak near hour 62 is similar for all.

Forecast maps. Each row uses the stored step with the biggest change, and the change maps use a symmetric range so 0 is in the middle. Columns 1 to 3 use `show_water`, columns 4 and 5 are change in `"RdBu"` (blue rising, red falling), column 6 is predicted minus true in `"PuOr"` (purple too much water, orange too little).

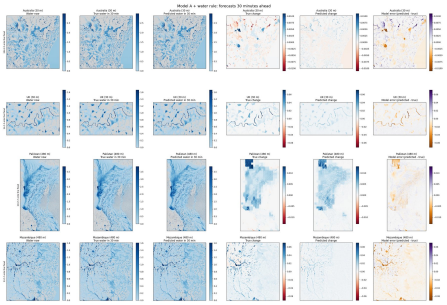


Figure 6. Model A with the water rule, 30 minutes ahead (Australia 204.0 h, UK 24.5 h, Pakistan 333.5 h, Mozambique 53.5 h). Columns: water now, true water in 30 min, predicted water, true change, predicted change, error (predicted minus true).

Say this: the first three columns look the same because water changes little in 30 minutes, which is why copying is strong. The predicted change is a fainter version of the true change, so the model finds rising water along rivers but underestimates it. The last three columns have their own scale per row (about ±0.01 m in Australia, ±0.06 m in the UK and Mozambique). The error is mostly orange along rivers in the UK and Mozambique and faintly purple over Australia.

4.9 Section 4: trend forecast and fitting β

The **trend forecast** lets the water keep changing as it just did, never below 0:

$$\text{next map} = \text{last map} + \beta \times (\text{last map} - \text{map 30 minutes before}).$$

β is one number fitted on the training part only, and there is no network. Let p be the change in the last 30 minutes and n the change in the next 30, so the trend guesses $n \approx \beta p$. The best β makes $\sum(n-\beta p)^2$ smallest (least squares), which gives

$$\beta = \frac{\sum pn}{\sum p^2},$$

with sums over every square and every training step. Example: $p = 1,2,3$ and $n = 1,2,2$ give $\beta = 11/14 = 0.786$.

```
    top = 0.0
    bottom = 0.0

    for step in range(first_target, last_target):
        last = region["depth"][step - 1]
        previous_change = (last - region["depth"][step - 2]).astype("float64")
        next_change = (region["depth"][step] - last).astype("float64")

```

```
        top += float(np.sum(previous_change * next_change))
        bottom += float(np.sum(previous_change ** 2))

    return top / bottom
```

Lines 3 and 4: the sums `top` and `bottom` start at 0. Lines 6 to 9: `previous_change` is p (map `step - 1` minus `step - 2`), `next_change` is n (map `step` minus `step - 1`), in `float64` for precision. Lines 11 and 12 add `np.sum(p * n)` to `top` and `np.sum(p ** 2)` to `bottom`, line 14 returns `top / bottom`.

β is fitted for each pair on its *training flood* (Australia, Pakistan) for steps `FIRST_TARGET` to `train_end`, so test parts and never-seen floods play no part. It prints `Trend numbers (beta): {'30m': 0.9207..., '480m': 0.9485...}`. So $\beta = 0.921$ (30 m) and 0.948 (480 m): about 92% and 95% of the last change carries on.

`score_simple_forecasts` (lines 17 to 44). Line 19: four running totals of squared error (copy, trend, copy + rule, trend + rule). Lines 21 to 24: `true`, `last`, `change`. Line 27: `past_rain = rain[step - 2]`, the same rain Model A sees. Lines 29 to 36 build the trend `np.maximum(last + trend_factor * change, 0)` and the four forecasts. Lines 38 to 44 add each squared error to its total, and skill is `1 - errors[name] / errors["copy"]`. The next cell runs it on the same test ranges as Model A, copies A's skills from the results table and saves `04_simple_forecasts`. All numbers are skills.

Region	Tested on	β	Trend	Trend+rule	Copy+rule	Model A	A+rule
Australia (30 m)	unseen time	0.921	0.990	0.990	0.000	−1.449	−0.710
UK (30 m)	never-seen	0.921	0.947	0.947	0.053	0.305	0.320
Pakistan (480 m)	unseen time	0.948	0.908	0.905	0.142	0.345	0.404
Mozambique (480 m)	never-seen	0.948	0.942	0.942	0.051	0.150	0.172

This is the key table. One fitted number and no network gives skill 0.990, 0.947, 0.908, 0.942, so it removes 91% to 99% of copying's squared error, while Model A reaches only −1.449, 0.305, 0.345 and 0.150. The trend is a fair opponent: β comes only from the training part of the training flood and the tests are the same as for A (the report says the comparison was checked in two ways). The rule adds almost nothing to the trend, because the last change already contains the rain. Copy with the rule alone is weak (0.000 in Australia, almost no rain late there, we think).

Q. What is β and how did you get it?
A. It is the share of the last 30-minute change that carries on. We fitted it by least squares as $\sum pn / \sum p^2$ on the training flood only. 0.921 for 30 m, 0.948 for 480 m.

4.10 Model B: `build_forecaster_v2`, `loss_in_cm`, zero check

The code calls Model B "version 2" or *v2*. Comment: it counts its correction in centimetres.

```
def build_forecaster_v2(trend_factor, height=TILE_SIZE, width=TILE_SIZE):
    # Model B: same layers as Model A, but it starts from the trend forecast

    inputs = keras.Input(shape=(INPUT_STEPS, height, width, CHANNELS), name="past_maps")

    # Same CNN + ConvLSTM + decoder as Model A
    x = layers.TimeDistributed(layers.Conv2D(16, 3, padding="same", activation="relu"), name="cnn_1")(inputs)
    x = layers.TimeDistributed(layers.Conv2D(16, 3, padding="same", activation="relu"), name="cnn_2")(x)
    memory = layers.ConvLSTM2D(32, 3, padding="same", name="memory")(x)
    x = layers.Conv2D(16, 3, padding="same", activation="relu", name="decoder")(memory)
    correction = layers.Conv2D(1, 1, kernel_initializer="zeros", name="correction")(x)

    # Trend forecast = last map + beta x last change (never below 0)
    last_depth = inputs[:, -1, :, :, 0:1]
    previous_depth = inputs[:, -2, :, :, 0:1]
    last_change = layers.Subtract()([last_depth, previous_depth])
    trend = layers.ReLU()(layers.Add()([last_depth, layers.Rescaling(float(trend_factor))(last_change)]))

    # Next map = trend + correction in centimetres (never below 0)
    next_depth = layers.ReLU(name="next_depth")(layers.Add()([trend, layers.Rescaling(CHANGE_UNIT)(correction)]))

    return keras.Model(inputs, next_depth, name="cnn_lstm_forecaster_v2")
```

Line 2: `CHANGE_UNIT = 0.01`, one unit of correction is 1 cm. Line 5: β is built into the model. Lines 11 to 15: input, CNN layers, ConvLSTM and decoder are exactly those of Model A. The last layer `correction` starts at zero and outputs centimetres. Lines 18 to 20: the newest depth (`-1`) and second newest (`-2`) are cut out and `layers.Subtract()` gives the last change. Line 21: `layers.Rescaling(float(trend_factor))` multiplies by the fixed β, `Add` puts it on the last depth and `ReLU` removes negatives. This `trend` is the trend forecast built from layers that learn nothing. Line 24: `Rescaling(CHANGE_UNIT)(correction)` multiplies by 0.01 (cm to m), adds it to `trend` and applies ReLU again. Forecast = trend + small correction. We think cm gives the network a convenient size, since in metres corrections are tiny. The report warns that starting point and unit were changed together, so it cannot say which mattered.

`loss_in_cm` divides the difference by `CHANGE_UNIT` (centimetres), `tf.square` squares it and `tf.reduce_mean` averages, so a mean squared error in cm^2. `v2_for_whole_map` (lines 34 to 38) is the whole-map copy for B.

The check (lines 42 to 51) builds an untrained Model B with the 30 m β, takes 50 checking tiles, computes the trend by hand in numpy (line 47) and compares it with `check_model.predict(sample, verbose=0)` (line 48). It prints `Parameters: 62833` (same as A, as Subtract, Rescaling, Add and ReLU learn nothing) and `Biggest difference from the trend forecast: 0.0`. The last layer starts at zero, so before training B is exactly the trend forecast.

Q. How does B differ from A?
A. Same layers. A adds its output (metres) to the last map, B adds its output (centimetres) to the trend forecast, so A starts as copying and B starts as the trend.

4.11 Load (or train) Model B, and test it

Same pattern as A (file `forecaster_v2_30m.weights.h5`, `compile` with Adam and `loss=loss_in_cm`, training only if `TRAIN_V2` is on or the file is missing). The new part is a safeguard.

```
        start_loss = float(np.ravel(forecaster.evaluate(
            data["validation_inputs"], data["validation_targets"], batch_size=32, verbose=0))[0])
        start_weights = forecaster.get_weights()
```

```
        # If training did not help, keep the plain trend forecast
        best_loss = min(history.history["val_loss"])
        if best_loss >= start_loss:
            forecaster.set_weights(start_weights)
```

Lines 15 to 17: `evaluate` measures untrained B on the checking tiles, which is the trend's error (`start_loss`), and `get_weights()` keeps the starting weights. Lines 19 to 25: same seed, early stopping and `fit` as A. Lines 28 to 30: if the best checking error is not below `start_loss`, the starting weights are put back, so B is never worse than the trend on the checking tiles. It prints `Loading Model B ...` and the same harmless warning. The report says training lowered the checking error only a little: about 1% for 30 m (0.1885 to 0.1868 cm) and 5% for 480 m (0.0243 to 0.0230 cm).

Testing Model B. For each pair the cell builds B at whole-map size, runs `evaluate_region` on the same test ranges, deletes the big model (`del`, frees memory), computes the trend's error per step for the plot, and saves `04_model_B_results`.

Region	Tested on	Trend	Model A	Model B	Model B + rule
Australia (30 m)	unseen time	0.990	−1.449	0.989	0.989
UK (30 m)	never-seen	0.947	0.305	0.948	0.949
Pakistan (480 m)	unseen time	0.908	0.345	0.908	0.901
Mozambique (480 m)	never-seen	0.942	0.150	0.943	0.941

Model B is within 0.001 of the trend in every test and far above Model A, but does not clearly beat the trend. The rule changes B by −0.007 to +0.001, so for a good forecast it adds nothing.

4.12 Comparison figures

Skill bars. Bars in orange (trend), blue (A) and green (B), values written by `ax.bar_label(..., fmt="%.3f")`, black line at skill 0 (`04_skill_comparison`).

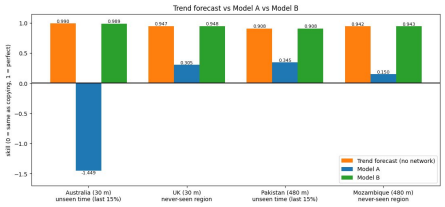


Figure 7. Skill of the trend forecast (orange, no network), Model A (blue) and Model B (green) in the four tests.

Say this: a simple trend does as well as Model B and far better than Model A, so at 30 minutes the network adds almost nothing (little is left to improve).

Error at every step. The same plot with the trend as a thick orange line and B as a green dashed line.

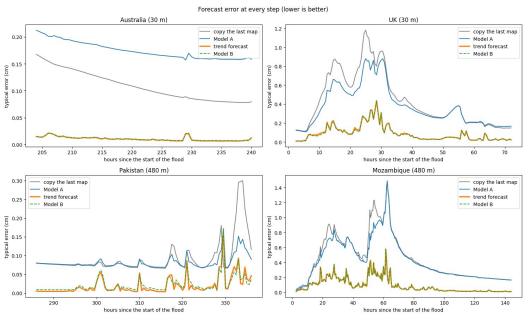


Figure 8. Typical error of each 30-minute forecast (lower is better). Grey: copy. Blue: Model A. Orange: trend. Green dashed: Model B.

Say this: orange and green lie almost on top of each other and far below grey and blue, so Model B follows the trend. In late Australia copying is 0.08 to 0.17 cm and Model A 0.16 to 0.21, while trend and B are about 0.01. The UK copy error peaks near 1.2 cm and the trend near 0.4, Mozambique 1.5 and 0.6. Exceptions: in Pakistan green sits slightly above orange in the calm early hours, and one spike near hour 329.

Error maps. Columns 2 to 4 share one scale taken from A's example, so colours are comparable. Errors are `predicted - true` without the rule.

Say this: errors sit where the water changes fastest, and Model B, which starts from the trend, has errors almost invisible at this scale. A has a faint purple tint over Australia, orange along rivers in the UK and Mozambique and an orange block at the top of Pakistan. B keeps only a thin orange line in the UK and small blocks elsewhere.

4.13 What this part shows

The notebook summary: at 30 minutes the trend removes 91% to 99% of copying's error, A stays far below it and B only matches it. A fair test of the LSTM's memory needs 6 to 24 hour forecasts (future work).

Q. Is your CNN-LSTM better than a simple method?
A. Not at 30 minutes. One fitted number gives a trend forecast as good as Model B and far better than Model A. A fair test needs 6 to 24 hour forecasts (future work).

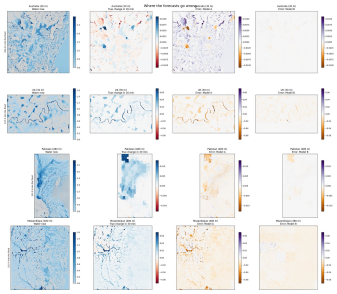


Figure 9. Where the forecasts go wrong. Rows: Australia, UK, Pakistan, Mozambique. Columns: water now, true change in 30 minutes, error of Model A, error of Model B. Purple is too much water, orange too little.

Q. What did you learn?
A. Compare a model with the best simple forecast, not only with copying.

4.14 Section 5: Capsule network and severity ratings (cells 42 to 44)

Where. Cell 42 is markdown, cells 43 and 44 are code and print nothing. From here every 64×64 tile gets one of four ratings for 30 minutes ahead. The rating is the share of the tile's normally dry land (rivers and lakes left out) under more than 30 cm of water, with limits `SEVERITY_LIMITS = [0.05, 0.15, 0.30]`. The markdown says "above 30%" for Severe, but `np.digitize` puts exactly 30% into Severe, as the report does.

Rating	None (0)	Minor (1)	Moderate (2)	Severe (3)
Dry land under more than 30 cm	below 5%	5% to 15%	15% to 30%	30% or more

The idea. The network sits on top of Model A and reuses its CNN and ConvLSTM, frozen. Each rating gets its own list of 16 numbers, a **capsule**. The **length** of the list is how sure the network is, and the answer is the rating with the longest list. The notebook gives no reason for 16.

Squash (cell 43, lines 3 to 4)

```
3     squared_length = tf.reduce_sum(tf.square(vectors), axis=-1, keepdims=True)
4     return squared_length / (1 + squared_length) * vectors / tf.sqrt(squared_length + 1e-7)
```

Line 3 squares every number and adds them along the last axis (`keepdims=True` keeps that axis), giving L, the squared length. Line 4 makes new vector $= \frac{L}{1+L} \times \frac{\text{vector}}{\sqrt{L}}$: the direction scaled to length 1, times a new length $L/(1+L)$. The `1e-7` avoids dividing by zero. A short vector stays short and a long one gets close to 1 but never reaches it, so every length is between 0 and 1 and reads as a confidence. Examples: $(0.3, 0.4)$ has length 0.5 and becomes length 0.2. $(3,4)$ has length 5 and becomes 0.962. $(30,40)$ has length 50 and becomes 0.9996. Lines 7 to 10 only wrap `squash` in a layer class `Squash`.

ClassCapsules: weights and routing by agreement

```
26        # One weight matrix for every (small capsule, class capsule) pair
27        self.vote_weights = self.add_weight(
28            shape=(number_of_inputs, self.number_of_classes, self.capsule_size, input_size),
29            initializer=keras.initializers.RandomNormal(stddev=0.05),
30            name="vote_weights")
```

`build` runs once, when the input (b, 256, 8) is known: 256 small (primary) capsules of 8 numbers. `add_weight` makes the learnable numbers, shape $(256, 4, 16, 8)$, a 16×8 matrix for every (small capsule, class) pair, $256 \times 4 \times 128 = 131{,}072$ weights (our own count), starting as small random numbers (`stddev=0.05`). A matrix turns 8 numbers into 16, a *vote*.

```
32    def call(self, primary_capsules):
33        # Every small capsule makes a vote for every class
34        votes = tf.einsum("pcoi,bpi->bpco", self.vote_weights, primary_capsules)
35
36        # At the start every vote counts the same
37        routing_scores = tf.zeros_like(votes[..., 0])
38
39        for round_number in range(self.routing_rounds):
40            # How much each small capsule sends to each class
41            coupling = tf.nn.softmax(routing_scores, axis=2)
42
43            # Class capsule = weighted sum of its votes, squashed
44            class_capsules = squash(tf.einsum("bpc,bpco->bco", coupling, votes))
45
46            # Votes that agree with the result count more in the next round
47            if round_number < self.routing_rounds - 1:
48                routing_scores = routing_scores + tf.einsum("bpco,bco->bpc", votes, class_capsules)
49
50        return class_capsules
```

In the einsum letters `b` is batch, `p` small capsule (256), `c` class (4), `o` number inside a class capsule (16), `i` number inside a small capsule (8). A letter left of `->` but missing on the right is added up.

- **Lines 32 to 34.** `"pcoi,bpi->bpco"`: **each of the 256 small capsules votes for each of the 4 classes**, 16 numbers per vote, shape (b, 256, 4, 16).
- **Lines 36 to 41.** `routing_scores` starts as zeros (b, 256, 4). The loop runs 3 times. `softmax` over the classes turns each small capsule's 4 scores into the **coupling**, positive shares adding to 1. Scores $(0,0,0,0)$ give 0.25 each, $(2,0,0,0)$ give $(0.711, 0.096, 0.096, 0.096)$.
- **Lines 43 to 44.** `"bpc,bpco->bco"`: each vote times its coupling, added over the 256, gives 4 class capsules of 16 numbers, then `squash`.
- **Lines 46 to 48.** If not the last round, `"bpco,bco->bpc"` takes the dot product of each vote with the class capsule just made (big positive means "my vote agrees") and *adds* it to `routing_scores`, so next round that small capsule sends more of its vote to the classes it agrees with. Line 50 returns the last round.

Think of 256 reporters writing for four judges. At first each report counts a quarter for each judge. Then each reporter sends more to the judges

whose note agrees with his. The notebook does not say why 3 rounds. We think it follows the capsule paper (Sabour and colleagues, cited in the report). This is general reasoning, not tested here.

CapsuleLength and the margin loss

```
53 class CapsuleLength(layers.Layer):
54     # Length of each class capsule = how sure the network is
55     def call(self, capsules):
56         return tf.sqrt(tf.reduce_sum(tf.square(capsules), axis=-1) + 1e-7)
57
58
59 def margin_loss(true_classes, lengths):
60     # The right class should be longer than 0.9, the others shorter than 0.1
61     correct = true_classes * tf.square(tf.nn.relu(0.9 - lengths))
62     wrong = 0.5 * (1 - true_classes) * tf.square(tf.nn.relu(lengths - 0.1))
63     return tf.reduce_sum(correct + wrong, axis=-1)
```

`CapsuleLength` squares the 16 numbers, adds them (the 16 disappear) and takes the root: input (b, 4, 16), output (b, 4), **four lengths per tile between 0 and 1**, the network's output. In `margin_loss`, `true_classes` is one-hot, for example $(0,0,1,0)$ for Moderate, and the comment gives the aim: right class above 0.9, others below 0.1. Line 61: `relu(0.9 - lengths)` is how far the right length is below 0.9, squared. Line 62: `relu(lengths - 0.1)` is how far a wrong length is above 0.1, squared, times 0.5 so wrong classes count half (in the capsule paper the reason is that absent classes should not shrink all lengths early on, general reasoning). Line 63 adds the four penalties. With true class Moderate, lengths $(0.05, 0.05, 0.95, 0.05)$ cost 0. Lengths $(0.05, 0.30, 0.80, 0.10)$ cost $0.1^2 + 0.5 \times 0.2^2 = 0.03$. Lengths $(0.90, 0.10, 0.10, 0.10)$ cost $0.8^2 + 0.5 \times 0.8^2 = 0.96$.

Q. What is routing by agreement, and why is the length the confidence?
A. Small capsules vote for every class and votes that match the result get more weight in the next of 3 rounds, while squash keeps lengths below 1 and the margin loss pushes the right class above 0.9.

Building the network on Model A (cell 44)

```
11     encoder = keras.Model(forecaster.inputs, forecaster.get_layer("memory").output, name="cnn_lstm_encoder")
12     encoder.trainable = False
13
14     inputs = keras.Input(shape=(INPUT_STEPS, TILE_SIZE, TILE_SIZE, CHANNELS), name="past_tiles")
15     memory = encoder(inputs)
```

Line 11: `forecaster.get_layer("memory").output` is the output of the ConvLSTM layer named `"memory"`, and `keras.Model(input, output)` makes `encoder`, **Model A cut off after the ConvLSTM**, with the weights Model A already learned. Output: 32 memory maps of 64×64. Line 12, `encoder.trainable = False`, **freezes** it. Lines 14 to 15 pass a new input through it to get `memory` (b, 64, 64, 32).

Why frozen? The notebook only says "frozen". Our reading is that Model A has already learned how water moves from 4,000 tiles, so the new layers only learn the rating and Model A's forecast stays unchanged. One sign agrees: the optimizer warning (cell 46) says the capsule optimizer has 16 variables, and Adam keeps 2 counters plus 2 per trained array, so $16 = 2 + 2 \times 7$ is 7 trained arrays, the new layers (3 convolutions with kernel and bias, plus the vote weights). Model A shows 24, $2 + 2 \times 11$. This is our own check.

```
17     # Shrink 64 x 64 -> 32 x 32 -> 16 x 16
18     x = layers.Conv2D(32, 5, strides=2, padding="same", activation="relu")(memory)
19     x = layers.Conv2D(32, 5, strides=2, padding="same", activation="relu")(x)
20
21     # Small capsules at 8 x 8 positions
22     x = layers.Conv2D(PRIMARY_TYPES * PRIMARY_SIZE, 3, strides=2, padding="same")(x)
23     x = layers.Reshape((PRIMARY_GRID * PRIMARY_GRID * PRIMARY_TYPES, PRIMARY_SIZE))(x)
24     primary_capsules = Squash(name="primary_capsules")(x)
```

Lines 17 to 19: two convolutions with 32 filters of 5×5, `strides=2` halves the map ($64 \to 32 \to 16$), `padding="same"` keeps only the stride changing the size. The notebook gives no reason for 5×5. Generally a stride-2 convolution shrinks the map and learns what to keep, and 5×5 sees more neighbours than 3×3. Line 22: $4 \times 8 = 32$ filters of 3×3, stride 2, no activation, giving 8×8 positions. Line 23: `Reshape((8 * 8 * 4, 8))` cuts the 32 numbers at each position into 4 pieces of 8, so $64 \times 4 = 256$ small capsules. Line 24 squashes them (**primary capsules**). Lines 27 to 33 add `ClassCapsules(4, 16, ...)` and `CapsuleLength`. `model` gives four lengths. `grad_cam_model` has the same layers with two outputs, the memory maps and the lengths, because Grad-CAM needs the memory. The 8 comes from `TILE_SIZE // 8`, three halvings.

Step	Shape	Meaning
Input tiles	(b, 4, 64, 64, 3)	4 maps, 3 layers
Frozen CNN + ConvLSTM	(b, 64, 64, 32)	32 memory maps
Conv 5×5 s2 twice, conv 3×3 s2	(b, 32, 32, 32), (b, 16, 16, 32), (b, 8, 8, 32)	map shrinks
Reshape + Squash	(b, 256, 8)	256 small capsules
ClassCapsules, CapsuleLength	(b, 4, 16), (b, 4)	class capsules, four lengths

4.15 Labels, class weights, loading and test tiles (cells 45 to 47)

Cell 45 (`tile_labels`) gives every tile its correct rating.

```
4     for number in range(len(targets)):
5         target_step, top, left = positions[number]
6         dry_land = region["dry_land"][top:top + TILE_SIZE, left:left + TILE_SIZE]
7         labels.append(severity_class(flooded_share(targets[number][:, :, 0], dry_land)))
8     return np.array(labels)
```

`positions[number]` is (time, top row, left column), `region["dry_land"][...]` cuts the dry-land mask of that piece, `targets[number][:, :, 0]` is the depth 30 minutes later, `flooded_share` gives the share of dry land deeper than 30 cm and `severity_class` turns it into 0 to 3. Lines 11 to 22 loop over both scales and count tiles per class with `np.bincount(labels, minlength=4)`.

	Training tiles (sum 4,000)				Class weight (our calculation)			
Model	None	Minor	Mod.	Severe	None	Minor	Mod.	Severe
30m	1942	1619	359	80	0.515	0.618	2.786	12.500
480m	234	562	1318	1886	4.274	1.779	0.759	0.530

In Australia (30 m) only 80 tiles (2%) are Severe. In Pakistan (480 m) 1,886 (about 47%) are, because a 480 m tile covers about 30 km against about 2 km at 30 m (report).

Cell 46 builds the capsule network on the trained Model A of each scale and loads saved weights. Nothing was trained in this run (`TRAIN_NEW = False`).

```
10      model.compile(optimizer=keras.optimizers.Adam(LEARNING_RATE), loss=margin_loss,
11                    metrics=[keras.metrics.CategoricalAccuracy(name="accuracy")])
```

```
16          # Rare classes count more, so the network can't just answer "None"
17          counts = np.bincount(data["train_labels"], minlength=4)
18          class_weights = len(data["train_labels"]) / (4 * np.maximum(counts, 1))
19          sample_weights = class_weights[data["train_labels"]]
20
21          stop_early = keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)
```

Lines 10 to 11: Adam with step 0.001, `loss=margin_loss` and an accuracy metric. Lines 16 to 19, **class weights**: `len(labels) / (4 * counts)` is $4000/(4 \times \text{count})$, and `class_weights[data["train_labels"]]` gives each tile its class weight (used as `sample_weight`), so rare classes count more. Each class then has equal total weight, for example $80 \times 12.5 = 1000 = 1942 \times 0.515$. Line 21: `EarlyStopping(patience=5, restore_best_weights=True)` stops after 5 epochs without a better checking loss and restores the best weights. The unused training branch is `model.fit` with one-hot labels, `epochs=30`, `batch_size=32`. The cell prints the loading message and the harmless optimizer warning (2 variables against 16 saved: the file also holds Adam's training memory, not needed for prediction). The same warning appeared for Models A and B.

Cell 47 cuts `TEST_TILES = 1000` random test tiles per flood (the last 15% in time of the training floods, and the whole never-seen UK and Mozambique floods). `predict` gives lengths (1000, 4) and Model A's forecast (1000, 64, 64, 1), and `np.argmax(lengths, axis=1)` is the **answer**, the longest capsule.

Be careful here

The 1,000 tiles are not independent. From map sizes we worked out the (time, tile) pairs (not printed): Australia $256 \times 73 = 18{,}688$, UK $8 \times 140 = 1{,}120$, Pakistan $72 \times 101 = 7{,}272$, Mozambique $4 \times 284 = 1{,}136$. In the UK nearly every pair is used and neighbours 30 minutes apart look almost the same.

Q. Why class weights?
A. The 30 m model has 1,942 None tiles and only 80 Severe, so Severe gets weight 12.5 and None 0.515, otherwise answering None would score well and teach little.

Q. Did the capsule network train in this run?
A. No, weights were loaded from Drive, the training code is up to 30 epochs with early stopping after 5 without improvement.

4.16 The confusion matrices (cell 48)

```
6       matrix = confusion_matrix(data["labels"], data["capsule_classes"], labels=range(4))
7       ConfusionMatrixDisplay(matrix, display_labels=SEVERITY_NAMES).plot(ax=ax, colorbar=False, cmap="Blues")
8
9       accuracy = np.mean(data["labels"] == data["capsule_classes"])
10      ax.set_title(REGION_NAMES[dataset] + "\naccuracy " + str(round(100 * accuracy, 1)) + "%")
```

Line 6: `confusion_matrix(true, answer, labels=range(4))` counts tiles, row i column j being true class i answered j. Line 7 draws it (darker blue is more tiles). Lines 9 to 10: `np.mean(labels == capsule_classes)` is the accuracy in the title. The other lines make 4 plots and save `05_capsule_confusion.png`.

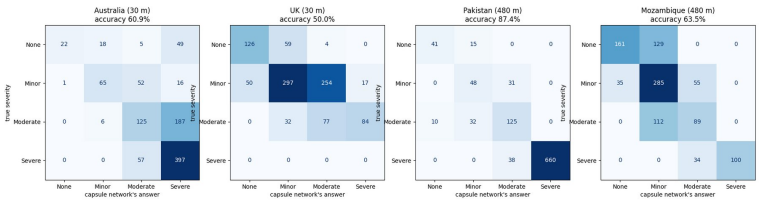


Figure 10. Capsule confusion matrices on 1,000 test tiles per flood (Australia, UK, Pakistan, Mozambique). Rows true rating, columns the answer, accuracy in each title.

Say this: the diagonal holds the correct answers and each matrix adds to 1,000. The overlapping y-label is only a layout problem.

Flood	True None	Minor	Moderate	Severe	Diagonal
Australia (30 m)	94	134	318	454	22+65+125+397 = 609
UK (30 m)	189	618	193	0	126+297+77+0 = 500
Pakistan (480 m)	56	79	167	698	41+48+125+660 = 874
Mozambique (480 m)	290	375	201	134	161+285+89+100 = 635

Titles: 60.9%, 50.0%, 87.4%, 63.5%.

- **Pakistan.** 660 of 698 truly Severe tiles (94.6%) were found. But 698 of 1,000 are Severe, so always answering "Severe" gives 69.8% and the 87.4% is less impressive than it looks.

- **UK.** The Severe row is all zeros because no UK test tile is truly Severe. 101 tiles were wrongly answered Severe. Of the 500 wrong answers, 479 are one class away.
- **Australia.** 649 tiles were answered Severe but only 454 are truly Severe. The 49 truly None answered Severe are unexplained (rule 1 also fires 49 times there).
- **Mozambique.** It never answers Severe for a tile below Moderate.

Q. How good is the capsule network?
A. Right for 50% to 87% of tiles, most mistakes are one class away, and Pakistan's high number mostly reflects that most of its tiles are Severe. The 30 m and 480 m accuracies should not be compared because the tiles cover different areas.

4.17 Section 6: Grad-CAM (cells 49 to 52)

Grad-CAM (explainable AI, the main guide) draws a heat map of which places mattered most for one number, in four steps. (1) Pick one number: for Model A the total predicted water, for the capsule network the length of the chosen class capsule. (2) Gradient: for each square of each of the 32 ConvLSTM memory maps, how much the number changes if that value goes up a little. (3) One weight per memory map: its gradient averaged over the map. (4) Multiply each map by its weight, add the 32, keep only the positive part.

```
5     with tf.GradientTape() as tape:
6         memory, output = grad_cam_model(model_input, training=False)
7         if class_number is None:
8             explained = tf.reduce_sum(output)
9         else:
10            explained = output[0, class_number]
```

Line 5: `tf.GradientTape()` records everything so gradients can be worked out later. Line 6: the model returns `memory` (1, height, width, 32) and `output`, with `training=False`. Lines 7 to 10: with no class number `tf.reduce_sum(output)` is the total forecast, otherwise `output[0, class_number]` is that capsule's length.

```
12    # How much the explained number changes when the memory changes
13    gradients = tape.gradient(explained, memory)
14
15    # One weight per memory map: its average gradient
16    channel_weights = tf.reduce_mean(gradients, axis=(1, 2))
17
18    # Weighted sum of the memory maps, keeping the positive part
19    heatmap = tf.nn.relu(tf.reduce_sum(memory * channel_weights[:, None, None, :], axis=-1))[0].numpy()
20
21    # Scale to 0 - 1
22    return heatmap / (heatmap.max() + 1e-8)
```

Lines 12 to 13: `tape.gradient(explained, memory)`, same shape as `memory`. Lines 15 to 16: `reduce_mean(..., axis=(1, 2))` gives (1, 32), one weight per map. Lines 18 to 19: `channel_weights[:, None, None, :]` makes shape (1, 1, 1, 32) so each map is multiplied by its own weight, `reduce_sum(axis=-1)` adds the 32, `relu` sets negatives to 0, `[0].numpy()` takes the single item. Lines 21 to 22 divide by `heatmap.max()` so the brightest is 1 (`1e-8` avoids division by zero). The heat map has the input's size because the memory maps do, so no resizing is needed.

The 256 windo

```
41    rows, columns = example["true"].shape
42    height = min(GRAD_CAM_WINDOW, rows)
43    width = min(GRAD_CAM_WINDOW, columns)
44
45    # Smooth the change map so the centre is a busy area, not a single square
46    change = np.abs(example["true"] - example["last"])
47    busy = uniform_filter(change, size=31)
48    centre_row, centre_column = np.unravel_index(np.argmax(busy), busy.shape)
49
50    top = int(np.clip(centre_row - height // 2, 0, rows - height))
51    left = int(np.clip(centre_column - width // 2, 0, columns - width))
```

For big maps the notebook zooms into a window of at most 256 × 256 (`GRAD_CAM_WINDOW = 256` on line 30). The cell says a whole map does not fit in GPU memory. We think it is because Grad-CAM keeps all 32 memory maps and gradients at once. Lines 41 to 43 give the window size, at most 256 each way. Lines 45 to 47: `uniform_filter(change, size=31)` averages the true change over 31 × 31 squares, so the busiest spot is a busy *area*. Line 48: `np.argmax` and `np.unravel_index` give its (row, column). Lines 50 to 51: the window starts at centre minus half its size and `np.clip(value, 0, rows - height)` keeps it inside the map. Windows (our calculation): Australia 256 × 256, UK 170 × 256, Pakistan 256 × 256, Mozambique 151 × 138 whole map.

Cell 51 runs Model A's Grad-CAM for each flood. It builds Model A at window size with `build_forecaster(height, width)`, copies the trained weights in with `set_weights(...get_weights())` (convolutions work at any size), and calls `grad_cam` with no class number.

```
15    model_input = make_input(region, example["step"] - 1, top, left, height, width)[None]
16    heatmap = grad_cam(grad_cam_model, model_input)
```

The drawing lines clip colours at the 99th and 99.5th percentiles and draw, per flood, water now, predicted change (blue rising, red falling) and the heat map in `inferno` (purple little, yellow most) at 60% opacity over grey ground height. **Cell 52** does the capsule network on the 4 most flooded tiles (`np.argsort(shares)[-4:]`) of the UK and Mozambique, with the network's *chosen* class (so it explains the capsule the network chose, not the correct one), the input depth (0 to 2 m) above and the heat below.

Say this (Model A): the heat is brightest where the predicted change is largest: the deep channel in Australia, thin lines along bends in the UK, square blocks in Pakistan (we think from blocks of heavy rain in the data, their straight edges are not how water behaves) and the drainage lines in Mozambique. It shows where the number is most sensitive, not that the forecast is right. The predicted change is tiny, about 1 to 3 cm in 30 minutes. The report adds that in the earlier laptop study deleting the top 10% of the heat map changed the forecast more than random cells in 84% to 92% of windows, not run in this notebook.

Say this (capsule): in the UK all four tiles read "true Moderate, network Severe" with heat on the deep channel. In Mozambique all four read "Severe, Severe" with spread-out heat. The four columns look alike and are very probably one tile at neighbouring times (we think). The report

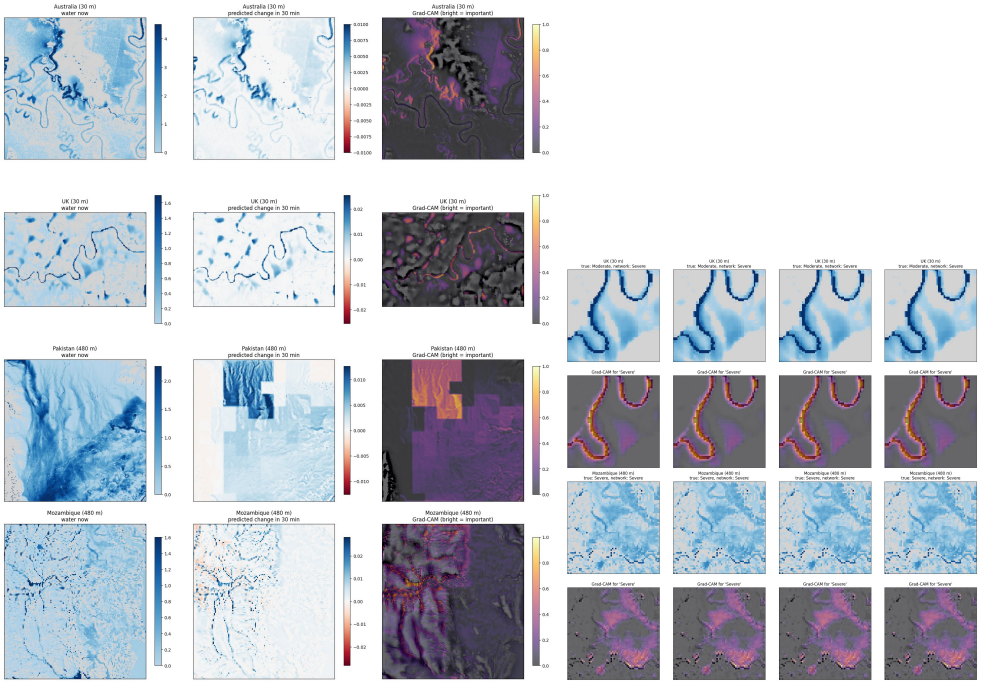


Figure 11. Left: Grad-CAM of Model A (rows Australia, UK, Pakistan, Mozambique, columns water now in m, predicted change in m, heat 0 to 1). Right: capsule network (rows 1 and 2 UK, rows 3 and 4 Mozambique, water now above its heat map).

says the capsule heat maps were weak (top cells beat two control sets in only 55% of windows, two seeds agreed with rank correlation 0.17), so they are illustrations, not proof.

Q. What number does each Grad-CAM explain, and why average gradients and use ReLU?
A. Model A: the total predicted water of the window. Capsule network: the length of the chosen class capsule. Averaging gives one weight per memory map and ReLU keeps only places that push the number up, the standard recipe.

4.18 Section 7: Written rules, the NeuroSymbolic part (cells 53 to 58)

NeuroSymbolic AI joins a network that learns patterns (Model A, capsule network) with rules people write down (these three plus the water-balance rule of Section 3a). The written reason of each decision is the explainable part.

Rule	When it applies	What it does	Why (our reading)
1. No water, no flood	under 1% of dry land flooded now and in the forecast	answer None	almost nothing wet now and soon makes a higher rating implausible
2. Rising water	forecast flooding at least as much as now, more than 1 mm/h rain in the last hour, network class below current class	answer is the current class	flooding not shrinking during rain should not be rated below what is already there
3. Agree with the forecast	network more than one class from the forecast map's class	answer is the forecast map's class	the forecast map is measured from depths, the network is a guess

The notebook gives no written reasons for the limits (1% and 1 mm/h), so the last column is our reading.

```
12      # Rule 1: no water, no flood
13      if share_now < ALMOST_DRY_SHARE and share_forecast < ALMOST_DRY_SHARE:
14          return 0, "Rule 1: under 1% of the land flooded now and in the forecast, so None"
15
16      # Rule 2: rising water during rain can't be less severe than now
17      if share_forecast >= share_now and rain_last_hour > RAIN_RULE_MM_PER_HOUR and network_class < now_class:
18          return now_class, "Rule 2: water rising during rain, so at least " + SEVERITY_NAMES[now_class] + " (the current level)"
19
20      # Rule 3: the network and the forecast map are more than one class apart
21      if abs(network_class - forecast_class) > 1:
22          return forecast_class, ("Rule 3: network said " + SEVERITY_NAMES[network_class] + " but the forecast map says "
23                                  + SEVERITY_NAMES[forecast_class] + ", so the forecast map wins")
24
25      return network_class, "No rule needed: the network's answer agrees with the rules"
```

`ALMOST_DRY_SHARE = 0.01` and `RAIN_RULE_MM_PER_HOUR = 1.0` are the limits, and `severity_class` turns the two shares into `now_class` and `forecast_class`. Lines 12 to 14, rule 1: `return 0, "Rule 1: ..."` ends the function at once. Lines 16 to 18, rule 2: three conditions joined by `and`, and `network_class < now_class` means rule 2 can only *raise* the answer. Lines 20 to 23, rule 3: `abs(network_class - forecast_class) > 1`. Line 25: no rule fired, "No rule needed". **Order matters**: the first rule that applies decides.

Cell 55 runs it for every test tile. Share now and share forecast come from the last depth map and Model A's forecast tile. Rain last hour is `data["inputs"][tile, -2:, :, :, 1].mean() * RAIN_SCALE`: the last two maps (two 30-minute steps), averaged, times 10 to undo the division when the input was built, giving mm per hour.

Cell 56 (accuracy with and without rules, saved as `07_neurosymbolic_accuracy.csv`) and **cell 57** (which rule decided, by cutting each reason at the colon):

Flood	Capsule alone	Forecast-map class	Capsule + rules	Changed	None needed	R1	R2	R3
Australia (30 m)	60.9%	99.8%	67.9%	7.0%	930	49	0	21
UK (30 m)	50.0%	99.1%	56.1%	6.1%	939	0	40	21
Pakistan (480 m)	87.4%	99.9%	89.1%	1.7%	983	0	7	10
Mozambique (480 m)	63.5%	99.5%	70.7%	7.2%	824	134	42	0

The gains (7.0, 6.1, 1.7, 7.2 points) equal the changed decisions (70, 61, 17, 72 tiles), and a change adds at most one correct tile, so every changed decision turned a wrong answer into a right one (our calculation). A rule that fired did not always change the answer: in Mozambique 134 tiles got rule 1 but rule 1 changed only 30 answers, in 104 the network already said None (our calculation). **Why the forecast map scores 99%:** the rating in 30 minutes is almost the rating now because water changes slowly, so the class read off the forecast map is nearly always right (the report says the classifier adds little). Pakistan's 87.4% is flattered by the 69.8% baseline.

Cell 58 shows ten example decisions (UK and Mozambique). For example UK 26.5 h, rule 3: network Severe, truth Minor, forecast map says Minor, $|3-1| = 2 > 1$, so Minor, correct. Mozambique 76.0 h had no rule, so the mistake stays. All six changed decisions of the ten are correct, but ten rows are an illustration, not a measurement.

Q. Did the rules help, and do they always give the right answer?
A. They helped in all four floods and every changed decision became correct, but they act only when their conditions are met, so not every mistake is fixed. Each decision carries a reason text starting with the rule name, and cell 57 counts these names.

4.19 Section 8: Summary and saved files (cells 59 to 61)

Cell 60 only copies numbers: for each flood `v2_table[v2_table["Region"] == REGION_NAMES[dataset]].iloc[0]` picks the Model B row, and the cell-56 accuracy row is placed beside it (saved as `08_summary.csv`).

Flood	Tested on	Trend	Model A	Model B	Capsule (%)	+ rules (%)
Australia (30 m)	unseen time	0.990	−1.449	0.989	60.9	67.9
UK (30 m)	never-seen region	0.947	0.305	0.948	50.0	56.1
Pakistan (480 m)	unseen time	0.908	0.345	0.908	87.4	89.1
Mozambique (480 m)	never-seen region	0.942	0.150	0.943	63.5	70.7

The first three numbers are skill = 1− (error of the forecast) / (error of copying the last map): 0 is as good as copying, 1 is perfect. Trend and Model B are close to 1 everywhere, Model A is far lower and negative in Australia, and nothing in the severity columns comes near the 99% of the plain forecast-map class.

Cell 61 lists what the notebook saved on Drive: 6 model files (all `.weights.h5`, only loaded in this run), 24 pictures and 15 tables. This notebook writes figures `02_` to `06_` and tables `01_`, `03_`, `04_`, `07_`, `08_`. Files such as `04_capsule_confusion.png`, `05_grad_cam_capsule.png`, `P1` to `P6` and `07_summary.csv` are written by no cell of this version. We think they are left over from earlier versions sharing the Drive folder, which is the answer if mam asks about a picture with another number. The only warning in this half is the harmless optimizer one at cell 46.

4.20 This notebook in ten lines

1. **Load the data.** Four floods from Drive: Australia and UK at 30 m, Pakistan and Mozambique at 480 m. A map step is half an hour and each input has 4 maps with 3 layers (depth, rain already fallen, ground height).
2. **Cut tiles.** Models learn from 4,000 random 64×64 tiles (600 for checking) from the first 70% and next 15% in time of Australia and Pakistan, and are tested on the last 15% of those and on the never-seen UK and Mozambique floods.
3. **Model A.** A CNN and a ConvLSTM with 62,833 parameters predict the change in depth, so next map = last map + change. Weights loaded from Drive.
4. **Results and water-balance rule.** Model A skill is 0.305, 0.345, 0.150 in the UK, Pakistan, Mozambique but −1.449 in Australia. The water-balance rule helps in all four.
5. **Trend forecast.** With no network, next map = last map + $\beta \times$ last change gives skill 0.990, 0.947, 0.908, 0.942 ($\beta = 0.921$ at 30 m, 0.948 at 480 m).
6. **Model B.** The same network starting from the trend and learning a small correction. Skill 0.989, 0.948, 0.908, 0.943, matching the trend, not beating it.
7. **Capsule network.** It reuses the frozen CNN and ConvLSTM of Model A and rates each tile None, Minor, Moderate or Severe with 256 small capsules, 4 class capsules and the length as confidence. Right for 60.9%, 50.0%, 87.4%, 63.5% of tiles.
8. **Grad-CAM.** Heat maps show where Model A looks (channels, flood edges, rain blocks) and where the capsule network looks (weak, only pictures).
9. **Written rules.** Three severity rules plus the water-balance rule are the NeuroSymbolic part. They raise severity accuracy to 67.9%, 56.1%, 89.1%, 70.7%, with a written reason for each decision.
10. **The summary.** At 30 minutes the plain trend forecast is as good as the networks and the forecast-map class is right about 99% of the time. A fair test of the ConvLSTM memory needs longer forecasts (6 to 24 hours), and all results are single runs on simulated floods.